SafeGuard AI: AI-Powered Personal Safety & Emergency Response App

By

Yow Yee Qin

FACULTY OF COMPUTING AND
INFORMATION TECHNOLOGY

TUNKU ABDUL RAHMAN UNIVERSITY OF
MANAGEMENT AND TECHNOLOGY
KUALA LUMPUR

ACADEMIC YEAR
2025/26

SafeGuard AI: AI-Powered Personal Safety & Emergency Response App

By

Yow Yee Qin

Supervisor: Ms Thamarai A/P Subramaniam

A project report submitted to the
Faculty of Computing and Information Technology
in partial fulfillment of the requirement for the
Bachelor of Software Engineering (Honours)

Faculty of Computing and Information Technology
Tunku Abdul Rahman University of Management and Technology
Kuala Lumpur

Copyright by Tunku Abdul Rahman University of Management and Technology.
All rights reserved. No part of this project documentation may be reproduced, stored in retrieval system, or transmitted in any form or by any means without prior permission of Tunku Abdul Rahman University of Management and Technology.

Declaration

The project submitted herewith is a result of my own efforts in totality and in every aspect of the project works. All information that has been obtained from other sources had been fully acknowledged. I understand that any plagiarism, cheating or collusion or any sorts constitutes a breach of TAR University rules and regulations and would be subjected to disciplinary actions.

Yow Yee Qin

Bachelor of Software Engineering (Honours)

ID: 24WMR09332

Abstract

SafeGuardAI is an AI-driven personal safety tool created to eliminate the critical gap in existing panic-button apps, which fail when a user is unconscious or immobilized. The main **purpose** of this project is to provide an automated emergency detection system that shortens response times and saves lives by actively recognizing 3 high-risk scenarios **falls** + **distress sounds (main detect)** and **inactivity (secondary detect)** all without needing any input from the user.

In the **scope** of functionality, SafeGuardAI includes a **fall-detection module** that uses an accelerometer to detect abrupt impacts and lack of movement. **Scream-detection** component utilizing a TensorFlow Lite model trained on selected audio samples. **Inactivity monitors** with customizable time thresholds. The application allows users to **set emergency contacts**, provide real-time GPS-based SOS alerts, and securely record all activities in Firebase.

The selected **methodology** uses **Flutter** for cross-platform development. The project utilizes **Firebase** services like Cloud Messaging for alert notifications and Firestore for saving event logs, based on feasibility during the development stage. The intended scream detection feature is being evaluated with **TensorFlow Lite.**

In the **testing** phase, the system will conduct **unit testing, integration testing,** and **system testing** with Katalon Studio to ensure all functions work as expected. These testing techniques aim to evaluate and improve the system's accuracy, reliability, and responsiveness before final deployment.

The expected **outcome** of this project is a functional and responsive mobile application to detect personal safety emergencies automatically. The system's **strengths** include cross-platform compatibility and detection through sensors and AI. However, anticipated **limitations** may include false detection, limited real-world testing, and the absence of direct integration with emergency services. Despite these challenges, the project establishes a strong foundation for upcoming enhancements and real-world deployment.

Acknowledgement

I want to express my sincere gratitude to my supervisor, Miss Thamarai A/P Subramaniam, for her continuous guidance, feedback, and support throughout my Final Year Project (FYP). Next, I want to thanks to my moderator, Dr Ho Chuk Fung for his helpful suggestions and support.

I want to give special thanks to my friends and family for their support throughout the whole FYP. Their understanding has given me strength to finish my FYP.

This project was completely independent, and I am truly grateful to everyone who has been part of this learning experience. Thank you for being a meaningful part of this achievement.

Table of Contents

1 Introduction 2

1.1 Problem Statement 3
1.2 Project Objectives 4
1.3 Project Background 5
1.4 Advantages & Contributions 6
1.5 Project Plan 7
1.5.1 Milestones 7
1.5.2 Gantt Chart 9
1.6 Scope Module 10
1.7 Chapter Summary and Evaluation 10

2 Literature Review 12

2.1 Review of Existing Personal Safety Applications 12
2.1.1 Comparison of Existing Personal Safety Applications 13
2.1.2 Advantages for SafeGuardAI with Existing Personal Safety Applications 14
2.2 Fall Detection 15
2.2.1 Threshold-Based Detection 15
2.2.2 Signal Magnitude Vector (SMV) 15
2.2.3 Application of Fall Detection 17
2.3 AI-based Scream Detection 18
2.3.1 Audio Capture & Preprocessing 18
2.3.2 Mel-Frequency Cepstral Coefficients (MFCC) 18
2.3.3 Support Vector Machine (SVM) 20
2.4 Inactivity Detection 22
2.4.1 Industry Practice: Galaxy Watch 12-15 seconds Stillness Window 22
2.4.2 SafeGuardAI Application 22
2.5 False Alarm Mitigation 23
2.5.1 Core Emergency Trigger (Fall + Scream) 23
2.5.2 Secondary Emergency Trigger (Fall / Scream) 23
2.5.3 Inactivity Detection as Confirmation, Not a Primary Trigger 23
2.6 Manual Trigger Recognition 24
2.6.1 Panic Button Activation (Long Press) 24
2.6.2 Shake Gesture Detection 24
2.7 Chapter Summary and Evaluation 25

3 Methodology and Requirements Analysis 27

3.1 Methodology 27
3.2 Requirement Gathering Techniques 28
3.2.1 Questionnaire 28
3.2.2 Unstructured Interview 28
3.3 Requirement Analysis (Including Diagrams) 29
3.3.1 Questionnaire 29
3.3.2 Unstructured Interview 34
3.3.3 Sequence Diagram 35
3.4 Functional and Non-Functional Requirements 37
3.4.1 Functional Requirements 37
3.4.2 Non-Functional Requirements 39
3.5 Chapter Summary and Evaluation 40

4 System Design 42

4.1 Use Case Design 43
4.2 UI Design 53
4.3 Data Design 64
4.3.1 Class Diagram 64
4.4 Process Design 66
4.4.1 Automatic Detection Activity Diagram 66

4.4.2 Manual Trigger Activity Diagram 68
4.5 Software Architecture Design 70
4.6 Security Design 72
4.7 Chapter Summary and Evaluation 73

5 Implementation and Testing 75

5.1 Sub-section 1 Heading 75
5.1.1 Sub-subsection Heading 75
5.2 Sub-section 2 Heading 75
5.3 Sub-section 1 Heading 75
5.3.1 Sub-subsection Heading 75
5.4 Sub-section 1 Heading 76
5.4.1 Sub-subsection Heading 76
5.5 Chapter Summary and Evaluation 76

6 System Deployment 78

System Backup and Risk Management 78
On-site Setup 78
Training Procedure 78
Follow-up 78
Chapter Summary and Evaluation 78

7 Discussions and Conclusion 80

Summary 80
Achievements 80
Contributions 80
Limitations and Future Improvements 80
Issues and Solutions 80

References 81

Chapter 1

Introduction

1 Introduction

This chapter presents an overview of the **SafeGuardAI** project, featuring the **problem statement, objectives, background,** and so on. It shows the purpose behind the development of a personal safety application and highlights the features that will be included, such as automatic fall detection, scream detection, inactivity monitoring, GPS-based alerting, and manual triggers like panic button and shake gesture. This project will concentrate more on mobile-based detection. Furthermore, it also introduces the project timeline and development responsibilities.

1.1 Problem Statement

University students who work as part-timers some stay in areas that lack direct bus service or MRT stations, forcing them to walk long distances through quiet and dimly streets to reach their destinations. This situation is **dangerous** when walking home alone late at night, when they are **incapacitated** by a fall, **medical emergency** or **physical attack,** they are unable to activate a **panic-button app manually,** which may hinder **timely assistance.**

In Malaysia, this weakness is exacerbated by increasing crime and injury statistics. Based on the Department of Statistics Malaysia (DOSM), the national Crime Index increased from 50,813 in 2022 to 52,444 in 2023, rising by 3.2%, reflecting a considerable increase in reportable crimes in all categories (JABATAN PERANGKAAN MALAYSIA DEPARTMENT of STATISTICS MALAYSIA, n.d.). At the same time, sexual offenses rose from 1,239 incidents in 2022 to 1,567 in 2023, reflecting a 26.5% increase, which cause greater trauma for victims, increased fear and anxiety within the community, and possible pressure on social and legal frameworks.

Existing smartphone safety apps primarily depend on **manual activation**, **inefficiency** method that can be **deadly** if users are incapacitated or unable to move. Moreover, single-mode detection (solely tracking button presses or basic motion thresholds) does not identify other emergencies such as audible distress signals or prolonged immobility.

However, this project is designed to tackle these issues by automatically identifying 3 high-risk situations: falls + distress noises (main detect) and inactivity (secondary detect) through on-device sensors (accelerometer, microphone) and efficient AI models. This approach allows the system to enable even impaired users to call for assistance, decreasing response times and possibly saving lives.

1.2 Project Objectives

The objectives of SafeGuardAI consist of 4 objectives to ensure a reliable, accurate, and user-centric emergency detection and alert system:

1. **Automated Multi-Modal Detection**
   Develop and validate an on-device model that combines fall, scream, and inactivity to automatically detect personal-safety emergencies with at least ≥ 90 percent detection accuracy.

2. **Minimize False Alarm**
   Implement a multi-trigger validation strategy which requires the **impact + sound (main detect) + inactivity (secondary detect)** followed by a 10-second confirmation prompt to minimize false alarm.

3. **Notifications & Manual Triggers**
   Ensure all confirmed emergencies such as (auto detection or manual triggers) that initiated via panic button or shake gesture trigger will push notifications through Firebase Cloud Messaging. Each notification will include live GPS coordinates, alert type (Fall, Scream, Inactivity, Manual).

4. **Intuitive User Control**
   Provide a simple settings interface that allows user manage 2 emergency contacts and an on/off auto-safety mode.

1.3 Project Background

Nowadays, smartphones are integral to daily life, yet Malaysia still lacks a truly proactive safety app that leverages their full sensor and computing capabilities. While other personal-safety apps require users to tap a panic button or call an emergency number, these **manual methods** fail when someone is **unconscious, immobilized, or panicked** and unable to interact with their device. Meanwhile, **Malaysia's Crime Index** has shown an increasing trend in recent years, with rising rates of violent and property crimes fueling public anxiety. Urban districts such as Kuala Lumpur and Johor Bahru report particularly **high incident volumes** during late-night hours when many workers and students make their commutes home.

However, there is currently **no Malaysian app,** and **very few global apps** that automatically **detects emergencies** whether an assault, medical crisis, or incapacitation by continuously monitoring the user's environment and physical state. Existing offerings remain reactive, relying on the user's ability to summon help rather than anticipating and identifying dangerous situations as they unfold.

SafeGuardAI was conceived to **fill this critical gap.** By harnessing built-in accelerometers and microphones, together with lightweight on-device AI models, the app can automatically detect high-risk events such as sudden impacts, distress noises, or prolonged immobility and dispatch alerts without any user action. This approach transforms the smartphone from a passive emergency tool into an active guardian. In doing so, SafeGuardAI offers Malaysia's first comprehensive, always-on **safety solution** tailored to both urban commuters and alone-working populations.

1.4 Advantages & Contributions

SafeGuard AI's design delivers several benefits:

1. **Multi-Modal Emergency Detection**

   Most existing apps just rely on 1 input detection. SafeGuardAI applies 3 different detection modules **(falls, distress sounds, and inactivity).** Then, only send alerting messages when (falls + distress sounds) as main detection, then add inactivity (as secondary detection) or more signals occur together.

2. **False Alarm Mitigation**

   To prevent accidental alerts, the app shows a **10 second confirmation pop-up** when a potential emergency is identified. If the user is safety and secure, they can **terminate** the notification. This feature helps **prevent false alarms triggered** by accidents like dropped phones or sudden loud noises, while still making sure user safety.

3. **Cross-Platform Accessibility**

   SafeGuardAI is developed by **Flutter,** which allows to operate on both **Android** and **iOS** devices from a single codebase. This guarantees broad accessibility for users regardless of phone brand and simplifies the updates and maintenance of the app across different platforms.

4. **Configurability & User Control**

   Provide **Auto-Safety mode,** users can turn it **on/off** for all emergency detection features according to their situations (turn on while walking home at night). Moreover, users are able to manage **2 emergency contacts.**

5. **Fast, & Reliable Alerting Notifications**

   Using **Firebase Cloud Messaging (FCM),** SafeGuardAI sends **real-time notifications** to **emergency contacts** when danger is detected. Each alert includes the type of alert, user's live GPS location, helping guards get alert when the user cannot communicate directly. (The Ultimate Guide to Firebase Cloud Messaging (FCM) - Novu, 2024)

1.5 Project Plan

1.5.1 Milestones

Project 1

No.	Task Name	Duration	Start Date	End Date
1	**Chapter 1: Introduction**	**7 days**	**23/6/2025**	**30/6/2025**
1.1	Problem Statement	2 days	23/6/2025	24/6/2025
1.2	Project Objectives	2 days	25/6/2025	26/6/2025
1.3	Project Background	1 days	27/6/2025	27/6/2025
1.4	Advantages & Contributions	1 days	28/6/2025	28/6/2025
1.5	Project Plan	1 days	29/6/2025	29/6/2025
1.6	Project Team & Organization	1 days	29/6/2025	29/6/2025
1.7	Chapter Summary & Evaluation	1 days	1/7/2025	1/7/2025
2	**Chapter 2: Literature Review**	**10 days**	**1/7/2025**	**14/7/2025**
2.1	Review of Existing Personal Safety Applications	2 days	1/7/2025	2/7/2025
2.2	Fall Detection	2 days	3/7/2025	4/7/2025
2.3	AI-based Scream Detection	2 days	5/7/2025	6/7/2025
2.4	Inactivity Detection	2 days	7/7/2025	8/7/2025
2.5	False Alarm Mitigation	2 days	9/7/2025	10/7/2025
2.6	Manual Trigger Recognition	2 days	11/7/2025	12/7/2025
2.7	Chapter Summary & Evaluation	2 days	13/7/2025	14/7/2025
3	**Chapter 3: Methodology & Requirement Analysis**	**11 days**	**15/7/2025**	**29/7/2025**
3.1	Methodology	3 days	15/7/2025	17/7/2025
3.2	Requirement Gathering Techniques	2 days	18/7/2025	19/7/2025
3.3	Requirement Analysis (Including Diagrams)	3 days	20/7/2025	22/7/2025
3.4	Functional & Non-Functional Requirements	3 days	23/7/2025	25/7/2025
3.5	Chapter Summary & Evaluation	3 days	26/7/2025	29/7/2025
4	**Chapter 4: System Design**	**16 days**	**29/7/2025**	**19/8/2025**
4.1	Use case Design	3 days	29/7/2025	31/7/2025
4.2	UI Design	3 days	1/8/2025	3/8/2025
4.3	Data Design	3 days	4/8/2025	6/8/2025
4.4	Process Design	2 days	7/8/2025	8/8/2025
4.5	Software Architecture Design	2 days	9/8/2025	10/8/2025
4.6	Security Design	2 days	11/8/2025	12/8/2025
4.7	Algorithm Design	2 days	13/8/2025	14/8/2025
4.8	Chapter Summary & Evaluation	2 days	17/8/2025	19/8/2025
	Project 1 Portfolio Submission	**-**	**-**	**8/9/2025**
	Initial System Preview with Supervisor	**-**	**-**	**26/9/2025**

Table 1.5.1: Project 1 Milestones

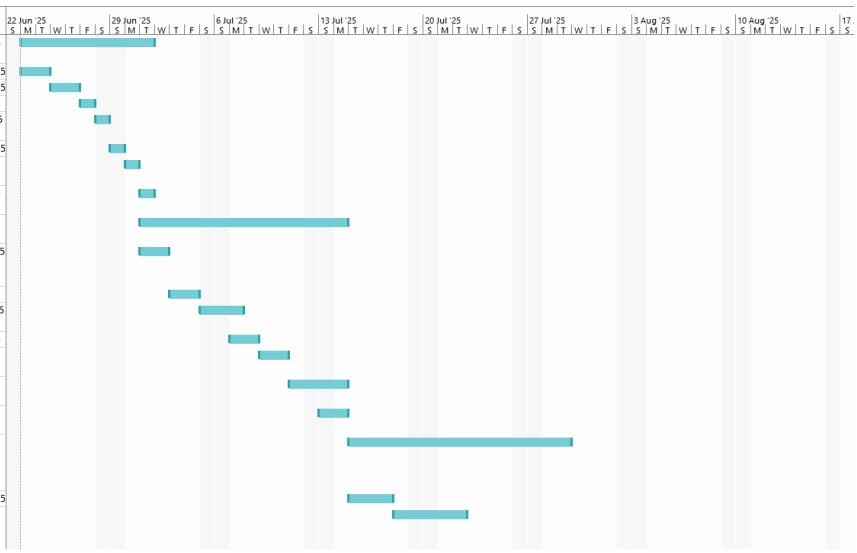
Project 2

No.	Task Name	Duration	Start Date	End Date
5	**Chapter 5: Implementation & Testing**	**30 days**	**29/9/2025**	**7/11/2025**
5.1	Implementation / Coding	10 days	29/9/2025	10/10/2025
5.2	Testing Strategies / Approaches	5 days	11/10/2025	15/10/2025
5.3	Test Plan	5 days	16/10/2025	20/10/2025
5.4	Test Data	5 days	21/10/2025	25/10/2025
5.5	Test Cases	5 days	26/10/2025	30/10/2025
5.6	Chapter Summary & Evaluation	5 days	1/11/2025	7/11/2025
	System Preview with Supervisor	**-**	**-**	**21/11/2025**
	Final System Testing with Supervisor & Moderator	**-**	**-**	**28/11/2025**
6	**Chapter 6: System Deployment (If Applicable)**	**7 days**	**8/12/2025**	**16/12/2025**
6.1	Deployment Environment	2 days	8/12/2025	9/12/2025
6.2	Deployment Process / Steps	3 days	10/12/2025	12/12/2025
6.3	Issues & Solutions	2 days	13/12/2025	15/12/2025
6.4	Chapter Summary & Evaluation	3 days	14/12/2025	16/12/2025
7	**Chapter 7: Discussions & Conclusions**	**5 days**	**15/12/2025**	**19/12/2025**
7.1	Summary	1 days	15/12/2025	15/12/2025
7.2	Achievements	1 days	16/12/2025	16/12/2025
7.3	Contributions	1 days	17/12/2025	17/122025
7.4	Limitations & Future Improvements	1 days	18/12/2025	18/12/2025
7.5	Issues & Solutions	1 days	18/12/2025	18/12/2025
7.6	Conclusions	1 days	19/12/2025	19/12/2025
	Draft FYP Report Submission	**-**	**-**	**12/12/2025**
	Final FYP Report Submission	**-**	**-**	**19/12/2025**

Table 1.5.2: Project 2 Milestones

1.5.2 Gantt Chart

ID	Number	Task Name	Duration	Start	Finish
1	**1**	**Chapter 1: Introduction**	**7 days**	**Mon 23/6/25**	**Tue 1/7/25**
2	1.1	Problem Statemen	2 days	Mon 23/6/2	Tue 24/6/25
3	1.2	Project Objectives	2 days	Wed 25/6/2	Thu 26/6/25
4	1.3	Project Background	1 day	Fri 27/6/25	Fri 27/6/25
5	1.4	Advantages & Contributions	1 day	Sat 28/6/25	Sat 28/6/25
6	1.5	Project Plan	1 day	Sun 29/6/25	Sun 29/6/25
7	1.6	Project Team & Organization	1 day	Mon 30/6/25	Mon 30/6/25
8	1.7	Chapter Summary & Evaluation	1 day	Tue 1/7/25	Tue 1/7/25
9	**2**	**Chapter 2: Literature Review**	**10 days**	**Tue 1/7/25**	**Mon 14/7/25**
10	2.1	Review of Existing Personal Safety Applications	2 days	Tue 1/7/25	Wed 2/7/25
11	2.2	Fall Detection	2 days	Thu 3/7/25	Fri 4/7/25
12	2.3	AI-based Scream Detection	2 days	Sat 5/7/25	Mon 7/7/25
13	2.4	Inactivity Detectior	2 days	Mon 7/7/25	Tue 8/7/25
14	2.5	False Alarm Mitigation	2 days	Wed 9/7/25	Thu 10/7/25
15	2.6	Manual Trigger Recognition	2 days	Fri 11/7/25	Mon 14/7/25
16	2.7	Chapter Summary & Evaluation	2 days	Sun 13/7/25	Mon 14/7/25
17	**3**	**Chapter 3: Methodology & Requirement Analysis**	**11 days**	**Tue 15/7/25**	**Tue 29/7/25**
18	3.1	Methodology	3 days	Tue 15/7/25	Thu 17/7/25
19	3.2	Requirement Gathering Techniques	3 days	Fri 18/7/25	Tue 22/7/25



ID	Number	Task Name	Duration	Start	Finish
19	3.2	Requirement Gathering Techniques	3 days	Fri 18/7/25	Tue 22/7/25
20	3.3	Requirement Analysis (Including Diagrams)	3 days	Sun 20/7/25	Tue 22/7/25
21	3.4	Functional & Non-Functional Requirements	3 days	Wed 23/7/25	Fri 25/7/25
22	3.5	Chapter Summary & Evaluation	3 days	Sat 26/7/25	Tue 29/7/25
23	**4**	**Chapter 4: System Design**	**16 days**	**Tue 29/7/25**	**Tue 19/8/25**
24	4.1	Use Case Design	3 days	Tue 29/7/25	Thu 31/7/25
25	4.2	UI Design	3 days	Fri 1/8/25	Tue 5/8/25
26	4.3	Data Design	3 days	Mon 4/8/25	Wed 6/8/25
27	4.4	Process Design	3 days	Thu 7/8/25	Mon 11/8/2
28	4.5	Software Architecture	3 days	Sat 9/8/25	Tue 12/8/25
29	4.6	Security Design	3 days	Mon 11/8/2	Wed 13/8/2
30	4.7	Algorithm Design	3 days	Wed 13/8/2	Fri 15/8/25
31	4.8	Chapter Summary & Evaluation	3 days	Sun 17/8/25	Tue 19/8/25
32	**0**	**Project 1 Portfolio Submission**	**1 day**	**Mon 8/9/25**	**Mon 8/9/25**
33	**0**	**Initial System Preview with Supervisor**	**1 day**	**Fri 26/9/25**	**Fri 26/9/25**

Figure 1.5.1: Gantt Chart Project 1

ID	Number	Task Name	Duration	Start	Finish
1	**5**	**Chapter 5: Implementation & Testing**	**30 days**	**Mon 29/9/25**	**Fri 7/11/25**
2	5.1	Implementation / Coding	10 days	Mon 29/9/25	Fri 10/10/25
3	5.2	Testing Strategies / Approaches	5 days	Sat 11/10/25	Thu 16/10/25
4	5.3	Test Plan	5 days	Thu 16/10/2	Wed 22/10/
5	5.4	Test Data	5 days	Tue 21/10/2	Mon 27/10/
6	5.5	Test Cases	5 days	Sun 26/10/2	Thu 30/10/2
7	5.6	Chapter Summary & Evaluation	6 days	Sat 1/11/25	Fri 7/11/25
8	**0**	**System Preview with Supervisor**	**1 day**	**Fri 21/11/25**	**Fri 21/11/25**
9	**0**	**Final System Testing with Supervisor & Moderator**	**1 day**	**Fri 28/11/25**	**Fri 28/11/25**
10	**6**	**Chapter 6: System Deployment (If Applicable)**	**7 days**	**Mon 8/12/25**	**Tue 16/12/25**
11	6.1	Deployment Environment	2 days	Mon 8/12/25	Tue 9/12/25
12	6.2	Deployment Process / Steps	3 days	Wed 10/12/25	Fri 12/12/25
13	6.3	Issues & Solutions	2 days	Sat 13/12/2	Mon 15/12/
14	6.4	Chapter Summary & Evaluation	3 days	Sun 14/12/25	Tue 16/12/25
15	**7**	**Chapter 7: Discussions & Conclusions**	**5 days**	**Mon 15/12/25**	**Fri 19/12/25**
16	7.1	Summary	1 day	Mon 15/12/	Mon 15/12/
17	7.2	Achievements	1 day	Tue 16/12/2	Tue 16/12/2
18	7.3	Contributions	1 day	Wed 17/12/	Wed 17/12/

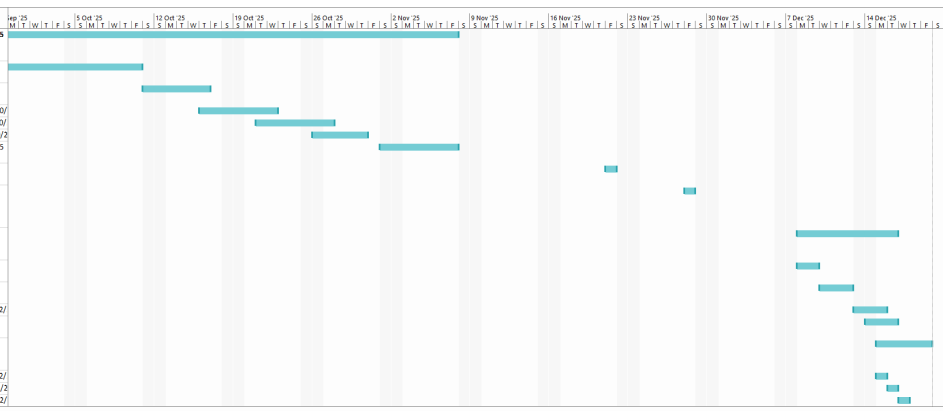


Figure 1.5.2: Gantt Chart Project 2

1.6 Scope Module

SafeGuardAI consists of multiple closely connected modules that work together to provide a continuous, reliable personal safety app. First module is **Fall Detection module** using accelerometer to detect sudden movement (threshold logic) and followed by post-impact stillness verification. Secondly, **Microphone-based Scream Detection (AI) module,** this module will capture short record audio clips, preprocessing it and use AI model training to recognize scream sound. Next, the **Inactivity Detection module** will detect when phone is not moving for long time via user toggle (auto-safety mode).

Furthermore, the **False Alarm Mitigation module** will reduce false alerts by using dual-trigger alerts (any 2 detection fall impact, sound, inactivity), also followed by a 10 second confirmation pop-up to minimize the accidental triggers. In addition, **Manual Trigger module** will provide a panic button UI and shake gesture for activation.

After an alert is confirmed, whether from automatically detection or manually triggers, the **GPS Integration & Notifications module** will obtain the user current locations and push notifications through Firebase Cloud Messaging (FCM) and store it into **database** to review the alert history.

Finally, the **UI/UX Design & Implementation module** will provide a user-friendly interface and the **Testing & Quality Assurance module** which include all the necessary testing such as unit testing, and so on.

1.7 Chapter Summary and Evaluation

This chapter introduces the **overall idea** and **purpose** of **SafeGuardAI.** It explained the real-life safety problems faced by individuals, especially when they are unable to call for help manually. The **objectives** of the project were clearly outlined, followed by background information that shows the **need for this system in Malaysia.** It also describes the key features and advantages of the system, including **automatic detection** and **quick alerting.** Furthermore, a detailed project timeline and scope module were also presented. Overall, this chapter sets the foundation for the next stages of the project by showing what the **system aims to solve** and **how it will be developed.**

Chapter 2

Literature Review

2 Literature Review

This chapter researches the primary algorithms and methods that are used in SafeGuardAI main module. It covers the compare existing apps, research threshold-based fall detection, AI-based scream detection, inactivity detection, false alarm mitigation, and manual trigger mechanisms. Together, these literature reviews laid the foundation of chosen algorithms and set clear targets.

2.1 Review of Existing Personal Safety Applications

Below is the comparison and what advantages for SafeGuardAI:

2.1.1 Comparison of Existing Personal Safety Applications

The 4 most widely used personal-safety apps: Noonlight, bSafe, Life360, and React Mobile share a common reliance on **manual activation** and **limited sensor use.** Noonlight connects directly to emergency services via a hold-to-release panic button and GPS tracking, but it doesn't offer automatic fall or scream detection and is only restricted to U.S. dispatch. While bSafe expands on live audio/video streaming and voice activation but still requires conscious user input and provides no AI-driven or real-time sensor-based alerts, making it ineffective if the user is incapacitated. Life360, while focus for family location sharing and vehicle crash detection, does not support fall, scream, or inactivity monitoring and is not tailored to pedestrian emergencies. React Mobile offers a simple one-touch panic alert and GPS link sharing but lacks any use of the phone's accelerometer or microphone and provides no automated emergency recognition. Together, these **limitations** highlight the need for **proactive, multi-modal detection** that can automatically detect falls, distress sounds, and prolonged immobility without requiring the user to trigger an alert manually.

App	Key Features	Limitations
Noonlight	• Hold-to-release panic button • GPS tracking • Connects to emergency service	• Require manual input • No automatic fall or scream detection • Limited to US emergency dispatch
bSafe	• Live audio / video streaming • Voice activation • GPS tracking & SOS alerts	• Relies on voice or button input • No AI / real-time sensor detection • Hard to trigger if unconscious
Life360	• Location sharing with family • Crash detection for vehicles • Driving & location history	• Crash detection limited to vehicles • No fall or inactivity detection • Not designed for walking or solo emergencies
React Mobile	• One-touch panic alert • GPS link sharing	• Fully manual system • No accelerometer / microphone usage • Lacks automatic emergency recognition

Table 2.1.1: Comparison of Existing Personal Safety Applications

2.1.2 Advantages for SafeGuardAI with Existing Personal Safety Applications

SafeGuardAI offers several clear advantages over existing personal-safety applications by combining multiple automated detection methods. The Automatic Fall Detection uses an accelerometer to recognize sudden impacts and immobility without requiring any button press. The AI-based Scream Detection module transforms microphone input into real time alerts when distress sounds occur, even if the user is incapacitated. Inactivity Monitoring will track prolonged stillness and then use Multi-Modal Detection Logic, which requires 3 concurrent signals before triggering an alert. To minimize false alarms, SafeGuardAI shows a 10-second confirmation popup, allowing accidental alerts to be easily canceled. However, for conscious users, they can trigger Manual SOS via a panic-button or discreet shake-gesture. Finally, SafeGuardAI is tailored explicitly for Malaysian use, that can use local emergency contacts and dispatch systems. Together, these features transform the smartphone into a proactive guardian rather than a passive safety tool.

SafeGuardAI Feature	**How It's Different**
Automatic Fall Detection	Uses phone sensors to detect, **not require user action**
AI-based Scream Detection	Detection distress sounds, **no button press**
Inactivity Monitoring	Detect when the **user is motionless** for too long
Multi-Modal Detection Logic	Combines **all danger signals** before trigger alert
False Alarm Prevention	Show a **10-seconds cancel popup** before sending alert
Manual SOS Triggers	Includes **panic button** and **shake gestures**
Designed for Malaysian Use	Support for **local emergency contacts**

Table 2.1.2: Advantages for SafeGuardAI with Existing Personal Safety

2.2 Fall Detection

SafeGuardAI use threshold-based rules for fall detection, while showing evidence using Signal Magnitude Vector (SMV) analysis. By recognizing the order **(free fall, impact, stillness),** it shows that fall happens.

2.2.1 Threshold-Based Detection

Event	Sensor Pattern (Acceleration)
Free Fall	Sudden drop in acceleration (SMV <0.5g)
Impact	Sharp spike in acceleration (SMV >2g)
Post-Fall Stillness	Phone remains nearly motionless after impact (SMV <0.3g for 8 seconds)

Table 2.2.1: Threshold-Based Detection

These values are based on experimental results. For example:

- **Impact forces** were measured at around **14–17 m/s²** (~1.4–1.7 g), followed by stillness over 8 seconds.
- Most systems use a **4-10 second stillness** window to ensure reliability.

2.2.2 Signal Magnitude Vector (SMV)

$$\text{Signal Magnitude Vector} = \sqrt{|A_x|^2 + |A_y|^2 + |A_z|^2}$$

Figure 2.2.1: Signal Magnitude Vector (SMV) formula

SMV simplifies the 3-axis accelerometer readings (X, Y, Z) into 1 combined value.

So that, we can use SMV **to identify a fall accident by deriving acceleration peak thresholds.** (Bicchielli, 2021)

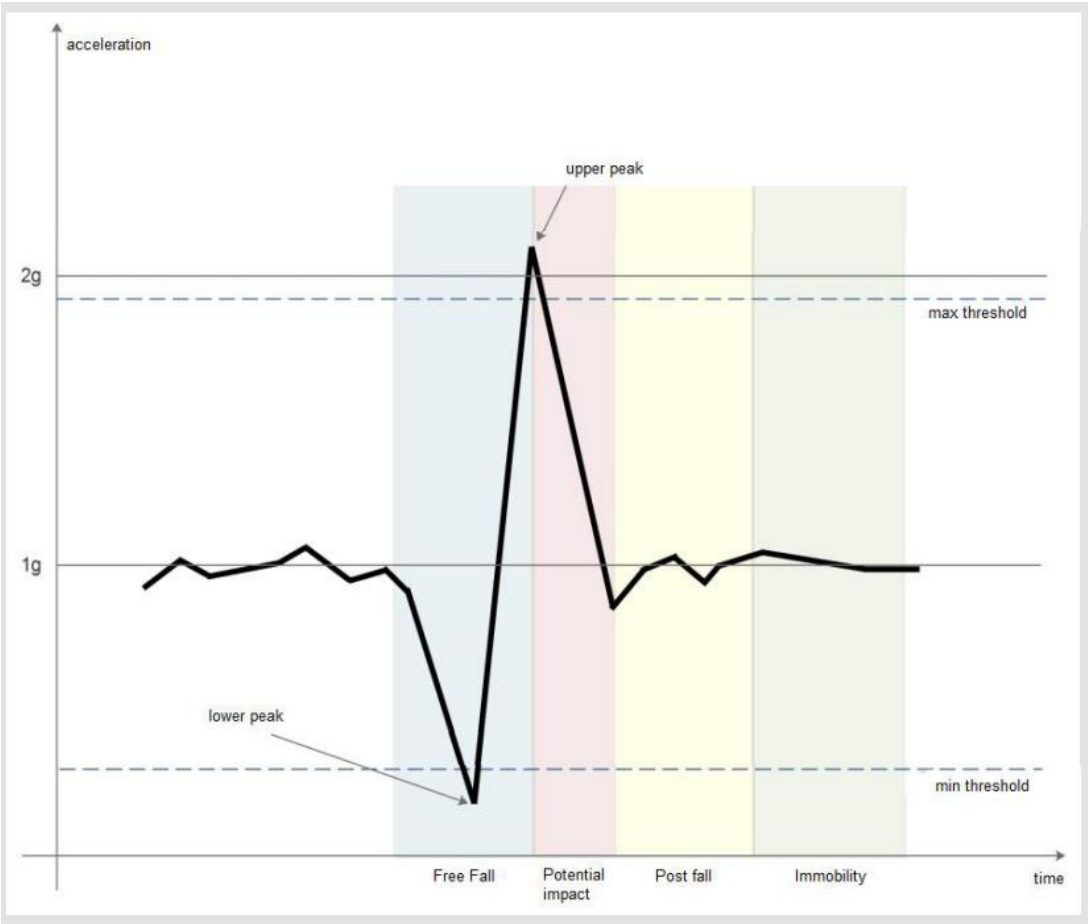


Figure 2.2.2: Evolution of the acceleration during a fall

1. **Free Fall (Blue Areas)**

   So, in freefall human and phone moving downward together, phone feel very little force **(lower peak near 0.5g, like floating)**

   Freefall is not that important to identify fall, but it can improve accuracy and reduce false alarms.

2. **Potential Impact (Red Area)**

   Sharp impact **(Upper Peak >2g)**, when person and phone hit the ground.

   Important to identify human is falling.

3. **Post fall & Immobility (Yellow & Green Area)**

   After a fall, humans may be unconscious or unable to move, and **accelerometers show almost no change for several seconds.**

   It is important to avoid false alarms.

2.2.3 Application of Fall Detection

In fall detection, I use the 3-phase threshold logic described in Section 2.2.1. These rules define when a fall is detected based on the pattern of motion, especially looking for high-impact spikes and followed by a period of stillness.

To support this logic, the Section 2.2.2 Signal Magnitude Vector (SMV) is used to support the evidence where I set the threshold logic. While ensuring it improves accuracy and reduces false alarms.

1. SMV falls below **0.5g** (start detect free-fall)
2. SMV above **2g** (impact detected)
3. SMV stay below **0.3g for 8s** (confirm stillness)

In summary, **trigger a fall alert only if all 3 happen in order**, reducing false alarms (dropped phone) and ensuring accurate detection.

2.3 AI-based Scream Detection

Scream detection will take raw audio clips turn into trustworthy emergency alerts through 3 synchronized phases. First, **audio capture & preprocessing** the audio clip into 25ms per frames and filter the background sounds before MFCC. Then, use **MFCC** to capture important sounds, that turn into 12-13 coefficients per frame. Finally, use **SVM** to recognize scream sounds. (IJRPR International Journal of Research Publication and Reviews, n.d.)

2.3.1 Audio Capture & Preprocessing

1. **Taking Sampling Audio**: Records **short audio** snippets (1s) via the phone microphone.

2. **Frame Segmentation**: Each audio clip is cut into many short pieces (around 25 milliseconds), and each piece overlaps a little with the one before it (by 10 milliseconds). This helps the system **catch quick sound changes** like sudden screams or loud noises.

3. **Noise Filtering & Normalization**: Ensure the audio clip **filters** the **background sounds,** removed or reduced the background sounds. Then, **adjust** each **audio clip sounds** to the **same level,** so that different loudness recordings don't confuse the system.

2.3.2 Mel-Frequency Cepstral Coefficients (MFCC)

Mel-Frequency Cepstral Coefficients **(MFCC)** is a way to turn raw microphone recordings (waveform) into small feature vectors that reflect how humans hear pitch and tone and **capture the important part of sounds** that can work for computer.

1. Mel Scale: Mel Scale is a non-linear scale used to measure pitch. Scream usually tends to have higher pitch, higher frequency, and faster time. In Mel Scale, lower frequencies (100 – 1,000 Hz) are spaced out more finely, while higher frequencies (above 1,000 Hz) are bunched together. So, Mel Scale is helpful to distinguish the difference on the spectrum, then making it easier for computers to spot scream patterns.

2. Cepstral Transformation: Used to convert Mel Scale spectrogram with log-scaled and Discrete Cosine Transform (DCT) to produce **12-13 coefficients per frame.** This is because 13 coefficients are a compressed signature sound in a short frame. Then, each coefficient captures different sound properties (energy, pitch pattern, and tone). Furthermore, each frame (13 coefficients per frame) is enough to **describe the important parts of the sound,** while ignoring the useless background sounds (raw audio).

3. MFCC Extraction Process (Vreča et al., 2024)

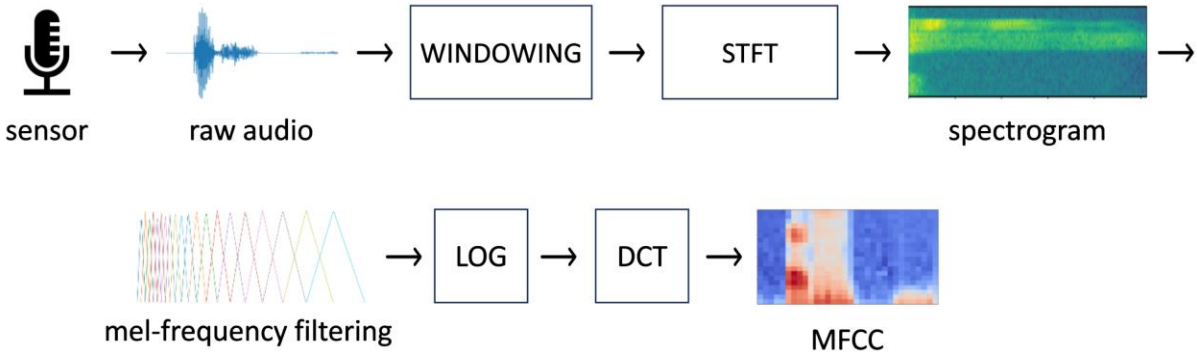


Figure 2.3.1: MFCC Extraction Process

This diagram shows the process of MFCC extraction. First, it processes the raw audio from the microphone into the frames (windowing) using Short-Time Fourier Transform (STFT) transform into spectrogram. Then, filter it with Mel Scale to distinguish the difference in pitch sounds. Then, it passed through log-scaled and DCT to produce MFCC. MFCC is useful as the input for the SVM classification model.

FYI: Scream sounds low parts around 100Hz (basic tone), but the sharp screeching part around 2000-4000Hz. This is because screaming starts at a low base, but it has many fast, high-pitched waves on top. Hence, MFCC works very well to detect and identify which is scream sound, as it captures both low tone and higher frequency shapes.

2.3.3 Support Vector Machine (SVM)

In the previous section, we transformed the raw audio into MFCC feature vectors. So, now we use Support Vector Machine (SVM) to distinguish scream and non-scream sounds. We plotted each short sound clip (MFCC 13) as a point in space. Some points are "scream" examples, others "non-scream" sounds. Then, **SVM finds** the **best** (line,2D / hyperplane,3D) that **separates scream points** from **non-scream points.**

1. **Collect & Label Data:** Collect MFCC vectors for 300 scream clips and 300 non-scream clips.

2. **Use RBF Kernel:** I will evaluate 2 methods to see which one works better for MFCC data.
   Linear Kernel: Draw straight line, simple and fast.
   Radial Basis Function (RBF) Kernel: Draw curve line, better in capturing complex patterns.

3. **Tune Hyperparameters:** We need to understand the C first. Depending on the data we need to evaluate to find the best C and best to distinguish scream and non-scream sounds clips.

   (What Is the Influence of C in SVMs With Linear Kernel?, n.d.)

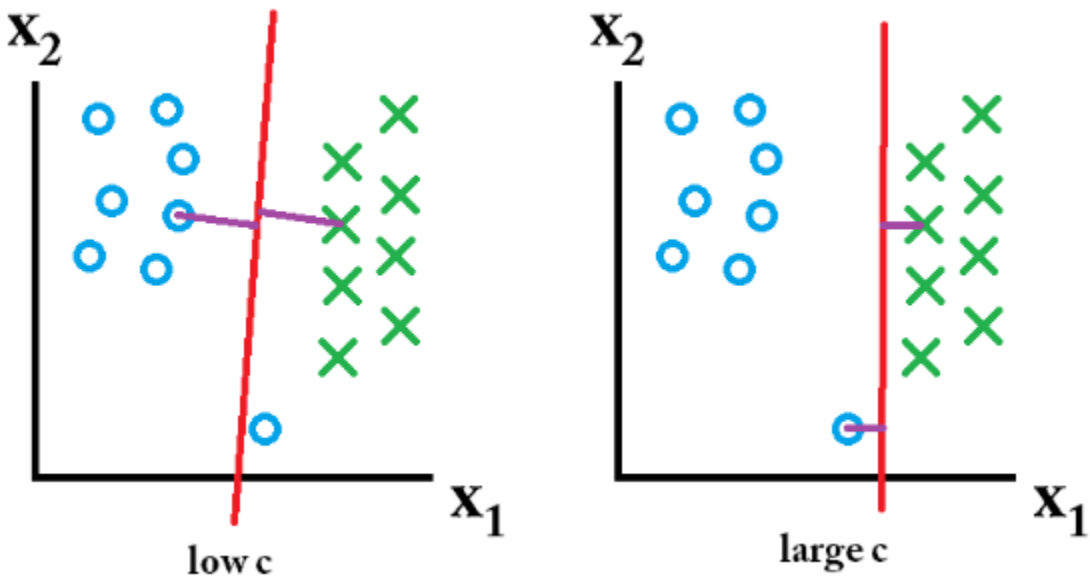


Figure 2.3.2: Comparison of low C vs large C

For now, we will use the common C first.

1. Linear SVM: C (Regularization) = 1
   Moderate value to balance a wide margin with reasonable fit to train data.
2. RBF SVM: C (Regularization) = 5
   Allow some misclassifications to achieve smoother curve lined.

4. **Target Performance Metrics:** During the SVM test, I try to hit the target metrics below.
   1. Precision ≥ 90%: Ensure at least 90% of scream alerts are true screams.
   2. Overall Accuracy ≥ 90%: Ensure total correct classifications (scream + non-scream) hit 90%.

2.4 Inactivity Detection

Inactivity detection acts as a secondary detector. It is important for confirmation step after a fall or scream and distinguish between real emergency and false alarm. (*Reddit - the Heart of the Internet*, 2021)

2.4.1 Industry Practice: Galaxy Watch 12-15 seconds Stillness Window

Commercial fall-detection wearables like the **Samsung Galaxy Watch** employ a short inactivity timeout following a detected fall. When the watch senses a hard impact, it immediately displays an on-screen alert and starts a **12–15 second** timer. If the user remains motionless for that entire window, the device assumes incapacity and proceeds to contact emergency services. However, **any significant movement** such as standing up or walking away during this interval **cancels the alert**, indicating that the wearer is conscious and unharmed.

2.4.2 SafeGuardAI Application

1. **Activation:** The inactivity timer starts only after a fall or scream is detected.

2. **Inactivity Duration: 12 seconds** of stillness.

3. **Movement Cancellation:** If accelerometer >0.1m/s² within the 12 seconds window, it will immediately stop the inactivity counting and dismiss the confirmation dialog.

4. **Confirmation Dialog:** If no movement occurs for the full 12 seconds, SafeGuardAI shows a 10-second confirmation popup, allowing the user to cancel a false alert. On the contrary, it will send the alert to emergency contacts.

By using the Galaxy Watch's **12–15 second inactivity** standards and cancellation logic, SafeGuardAI ensures fast and reliable confirmation of incapacitation without unnecessary delays or false alarms.

2.5 False Alarm Mitigation

Minimizing false alarms is important to maintain user trust and ensure that emergency contact takes it seriously. Therefore, SafeGuardAI is designed with a 3-tiers false alarm mitigation strategy and shows a 10 second confirmation pop-up dialog for every potential alert. Users need to cancel it to avoid accidental alerts.

2.5.1 Core Emergency Trigger (Fall + Scream)

When both **fall + scream** is detected within **5 seconds** windows, SafeGuardAI will immediately shows a 10 second confirmation pop-up, allowing conscious users the opportunity to cancel the alert. If the cancellation is not received, emergency contact will receive and alerted with user's current location.

2.5.2 Secondary Emergency Trigger (Fall / Scream)

When a single trigger is identified, either fall or scream, the system starts a 12 second inactivity countdown.

1. **Fall only:** Start inactivity detection. If there is no movement for 12 seconds, then show a confirmation dialog.
2. **Scream only:** When 2 or more screams are detected within 5 seconds, it immediately shows a confirmation dialog. However, only 1 scream was detected, 12 seconds inactivity countdown started, then show a confirmation dialog.

This method allows SafeGuardAI to detect less obvious emergencies alerts such as **(fainting without noise or verbal distress sound without impact).** It also helps avoid waiting unnecessarily for all 3 triggers **(fall, scream, inactivity)** before taking any action.

2.5.3 Inactivity Detection as Confirmation, Not a Primary Trigger

Inactivity detection is treated as confirming signal rather than a primary trigger. It only checks after a fall or scream is detected. Because it prevents false alerts when the phone is left in the bag, not in used, user is sleeping, and so on. In other words, stillness alone is never enough to trigger an emergency alert.

2.6 Manual Trigger Recognition

Manual triggers provide a dependable option for aware users to request assistance immediately. SafeGuardAI offers two supplementary manual features: a **panic button** on the screen that requires a 5-**second-long press,** and a **shake gesture sensor** that utilizes accelerometer data. Both approaches have been confirmed in mobile UX and gesture-sensor studies as efficient, low-latency alert initiators

2.6.1 Panic Button Activation (Long Press)

Panic buttons need a long press to avoid accidental taps. According to a study, Android 's Accessibility guidelines specify "touch & hold" (long press) delays between 500ms to 1,500ms (0.5s - 1.5seconds) to distinguish between intentional holds from simple accidental taps. (Touch & Hold Delay - Android Accessibility Help, n.d.). However, **SafeGuardAI** will **set 5 seconds long press** to ensure only deliberate press triggers the alert, which is still a reliable, and fast way to call for help.

2.6.2 Shake Gesture Detection

Shake detection uses the built in accelerometer in phone to detect shake, back-and-forth movements, providing a hands-free and discreet activation method.

1. **Thresholding:** Monitor Signal Magnitude Vector (SMV) and mark a sample when SMV exceeds a set threshold (2 m/s^2) to detect strong motions.
2. **Peak Counting:** Within a short time frame (3 seconds), count the number of above-threshold peaks. For example, if 5 shakes/ peaks occur within 3 seconds, a shake gesture is confirmed.

2.7 Chapter Summary and Evaluation

Chapter 2 has established a solid theoretical foundation for SafeGuardAI multi-modal detection. It surveys the existing apps and the advantages of SafeGuardAI. Furthermore, research the fall detection model, AI-scream detection, and inactivity detection. Moreover, it also studies the framework for false-alarm mitigation to minimize false alarms and manual triggers mechanism to allow conscious users manually trigger the alert for help. In the nutshell, this research helps validate the selected algorithms to be used in chapter 3 later.

Chapter 3

Methodology and Requirements Analysis

3 Methodology and Requirements Analysis

This chapter outlines the methodology and requirement analysis for SafeGuardAI. It started with an incremental development model. Furthermore, using requirement gathering techniques like questionnaires and unsupervised interviews to gather user requirements and expectations. Moreover, it also shows the functional and non-functional requirements that must be achieved. Finally, sequence diagram is being used to illustrate the interaction between different modules.

3.1 Methodology

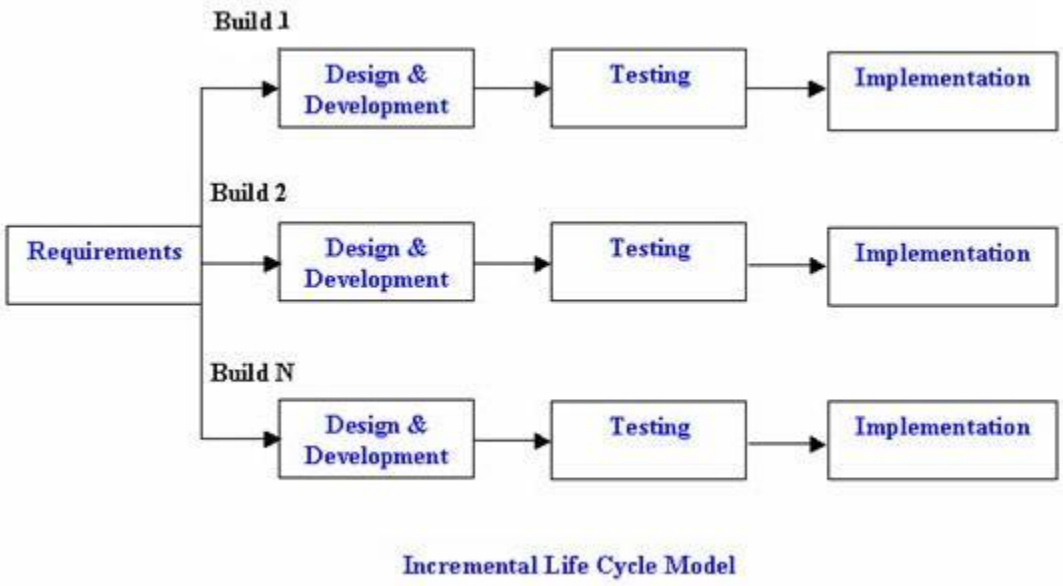


Figure 3.1.1: Incremental Development Model (*Try QA*, n.d.)

This project will use an incremental development model, as it provides the SafeGuardAI in a cycle of iterative updates. This model fits the system because it combines several distinct technical components sensor processing, real-time decision logic, and on-device audio classification, whose feasibility and performance are uncertain until prototyping and testing are performed on actual devices. By developing small, testable modules (for example, an accelerometer-based fall detector, a manual-trigger UI, then the scream classifier and finally the fusion/notification layer) the project reduces risk by each increment produces a demonstrable deliverable that can be validated, tuned, and reviewed by the supervisor. This model also enables early usability feedback (from questionnaires and peer tests) to influence later increments, supports progressive integration of the ML component once sufficient audio data are collected, and confines complex work such as on-device inference until the surrounding system is stable. Then, for the quality assurance and testing, every incremental will be gone through unit tests and integration tests. In general, the incremental model balances well with speed, safety and flexibility, making SafeGuardAI deliver a reliable core system.

3.2 Requirement Gathering Techniques

To understand user needs and user expectations, 2 methods of requirements gathering techniques have been conducted.

3.2.1 Questionnaire

A structured questionnaire has been conducted through Google Forms, designated to be completed within 2-3 minutes. The main target is university students and young workers who commute at night. The questionnaire includes demographic questions and requirement-focused questions. This method was chosen because of the efficiency of collecting large quantitative data in a short time, and the results can be analysis in the pier charts.

3.2.2 Unstructured Interview

An unstructured interview has been conducted with some selected participants from the target audience. Unlike questionnaires, unstructured interviews allow the target audience to freely express their thoughts, concerns, and suggestions about the safety app features. This method was chosen because of the efficiency of collecting real qualitative data, as it comes from their personal real experience.

3.3 Requirement Analysis (Including Diagrams)

3.3.1 Questionnaire

Based on the 32 responses, several key questions will be analyzed here. Figure (Pie Chart) will also be inserted here to illustrate results.

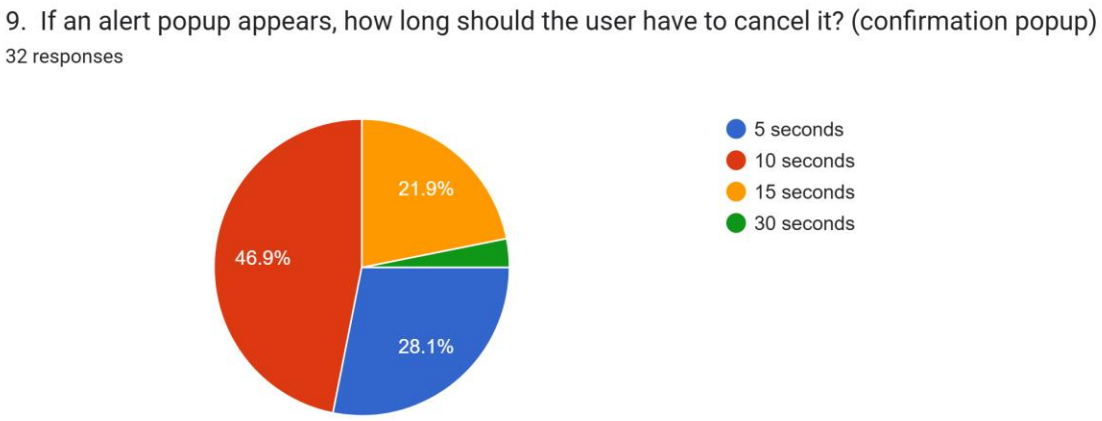


Figure 3.3.1: Confirmation Popup Duration

Based on the figure above, almost half of respondents (15/32, 46.9%) preferred a **10-second confirmation popup,** while others leaned toward shorter or longer durations. This confirms that 10 seconds is a balanced default, giving enough time to cancel false alarms without delaying responses in real emergencies.

10. If only one event occurs (a single loud scream or a single fall), how long should the app wait before checking for inactivity/confirming?
32 responses

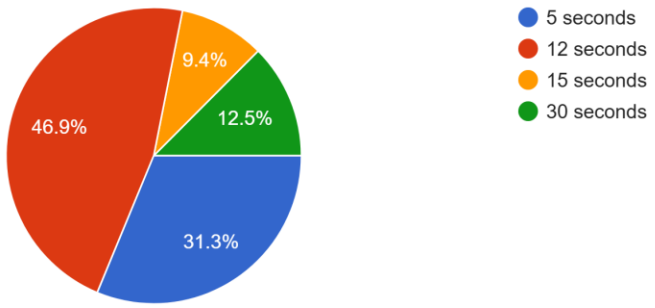


Figure 3.3.2: Inactivity Checking

Based on the figure above, almost half of respondents (15/32, 46.9%) preferred a **12-second inactivity check.** Respondents felt this was long enough to verify true immobility while still being responsive. This supports setting 12 seconds as the default threshold for inactivity confirmation.

11. If two or more screams occur within 5 seconds, should the app show the confirmation popup immediately?
32 responses

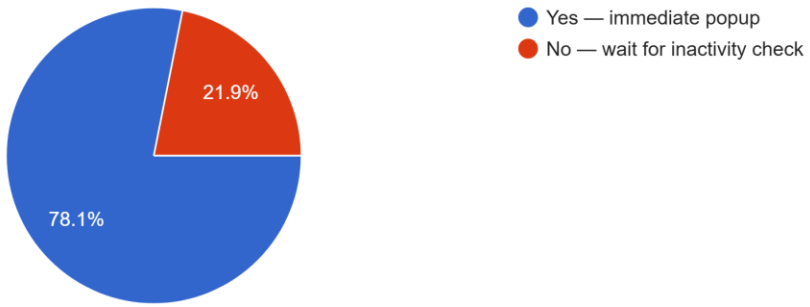


Figure 3.3.3: Scream Detection Rule

Based on the figure above, the majority (25/32, 78.1%) supported **triggering immediate** action if **two screams or more occur within 5 seconds.** This validates the dual-scream detection rule, ensuring rapid detection of vocal distress while reducing false alarms caused by random background noise.

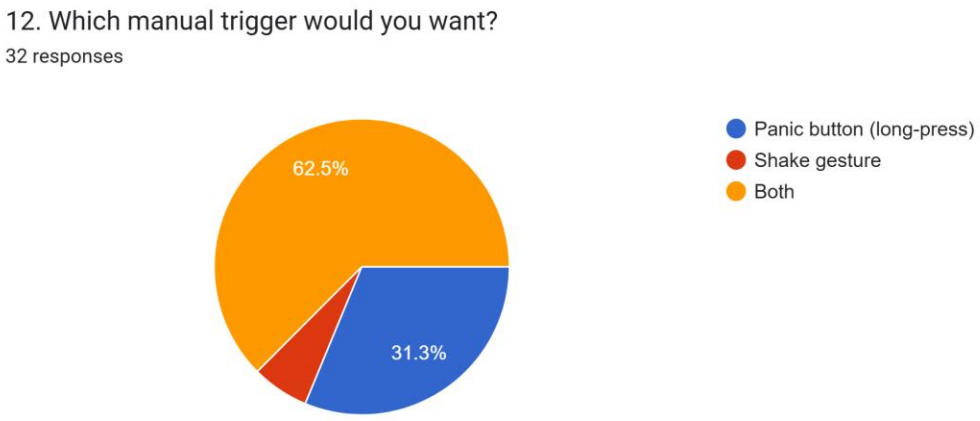


Figure 3.3.4: Manual Trigger Preferences

Based on the figure above, most respondents (20/32, 62.5%) preferred having **both panic button and shake gesture** available as manual SOS options. This indicates users want flexibility: the panic button for reliability, and the shake gesture as a discreet backup when hands are occupied.

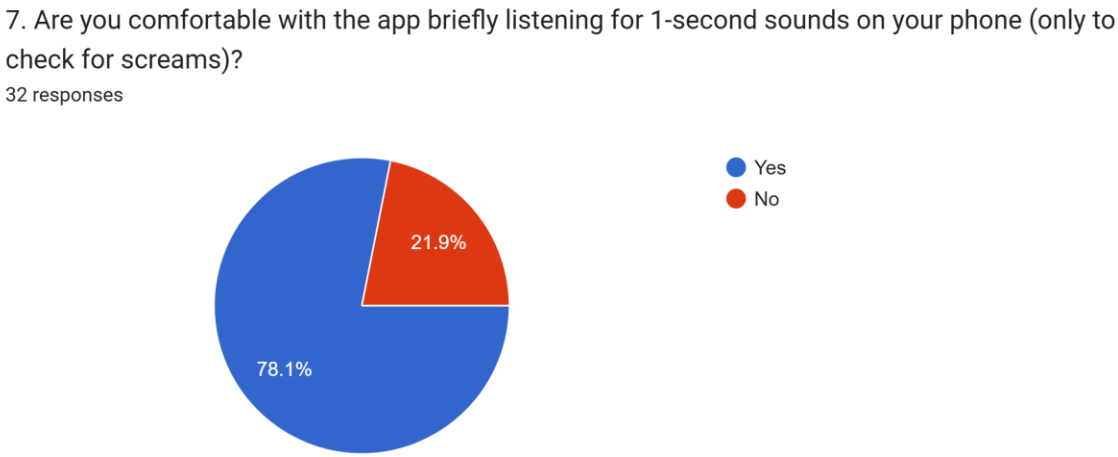


Figure 3.3.5: Audio privacy

Based on the figure above, a large majority (25/32, 78.1%) agreed to allow **short 1-second audio snippets processed only on the phone**. This confirms strong support for on-device processing as long as audio is not uploaded, aligning with privacy requirements in system design.

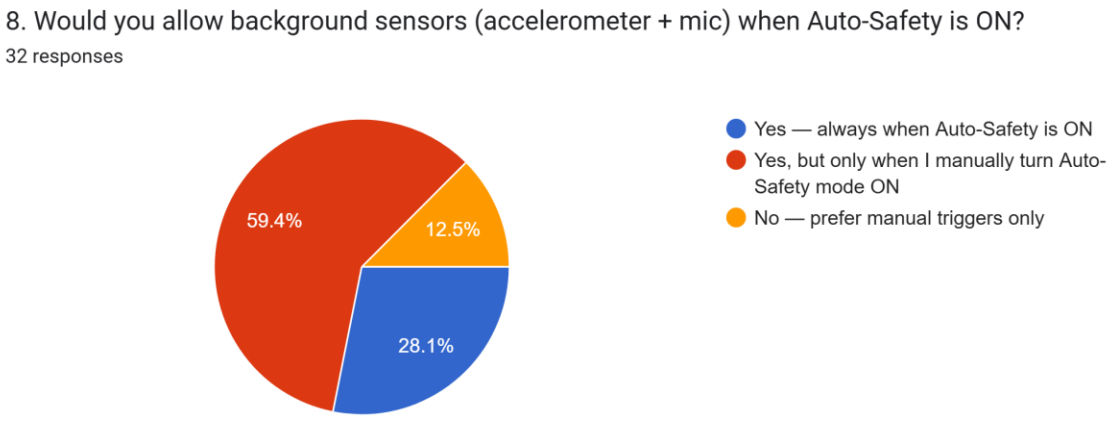


Figure 3.3.6: Auto-Safety Mode Preference

Based on the figure above, over half (19/32, 59.4%) preferred Auto-Safety to be **manually enabled** rather than always-on. This suggests the system should default to user-controlled activation, giving users the choice to turn on monitoring only during risky commutes.

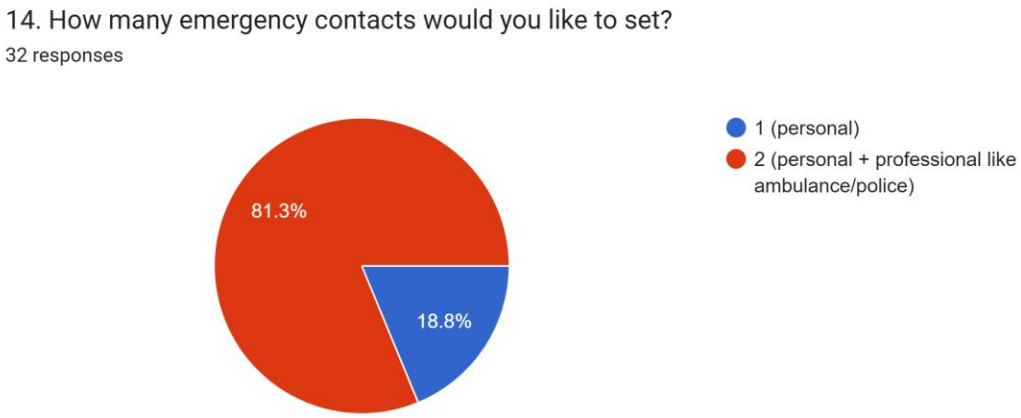


Figure 3.3.7: Emergency Contacts

Based on the figure above, most respondents (26/32, 81.2%) supported limiting emergency contacts up to two. This ensures that notifications are manageable and reliable, avoiding confusion caused by too many contacts.

Questionnaire analysis summary

The questionnaire validates SafeGuardAI design choices: **10-second confirmation, 12-second inactivity timeout, dual-scream trigger, combined manual triggers, on-device audio processing**, **user-controlled Auto-Safety,** and **2 emergency contacts.** These results

form the foundation for functional requirements by combining majority preferences with practical implementation feasibility.

3.3.2 Unstructured Interview

The unstructured interviews provide deeper insights than the questionnaire. 5 key themes shows below:

1. Privacy & On-device Preprocessing:
   Participants consistently stressed that audio should never be uploaded. They would be comfortable with local processing if privacy was guaranteed. Example response: “Don’t upload my audio, please just process it on my phone.”
2. Confirmation Popup Timing:
   While the survey favored 10 seconds, interviews revealed why: participants feared false alarms from accidental drops but wanted a short enough delay for emergencies.
   Example response: “If the phone gets dropped, a 10-second popup is useful; 15 s is too long.”
3. Manual Triggers
   Both panic button and shake gestures were requested. However, several participants suggested a test mode to check shake sensitivity, reducing accidental alerts. Example response: “I'm prefer to have both manual triggers.”
4. Multi-Modal Detection
   Users support fast escalation when multiple signals occur together (fall + scream or repeated screams). This validates the proposed multi-trigger fusion logic. Example response: “Multi detection is important than only have one detection.”
5. User Control
   Many said Auto-Safety should not always run but be enabled when needed. This matches survey results where most wanted manual toggling. Example response; “The system must allow me to freely enable or disable the auto safety mode.”

Unstructured interviews analysis summary

The interviews strengthened confidence in the questionnaire findings while adding qualitative context. Especially, they highlighted privacy as a central concern and supported 10 seconds as a practical confirmation time.

3.3.3 Sequence Diagram

Sequence diagrams are important because they provide a clear visualization of how different components of SafeGuardAI interact over time. As it shows the order of messages, activations, and decision flows, to understand how emergency detection leads to a final notification.

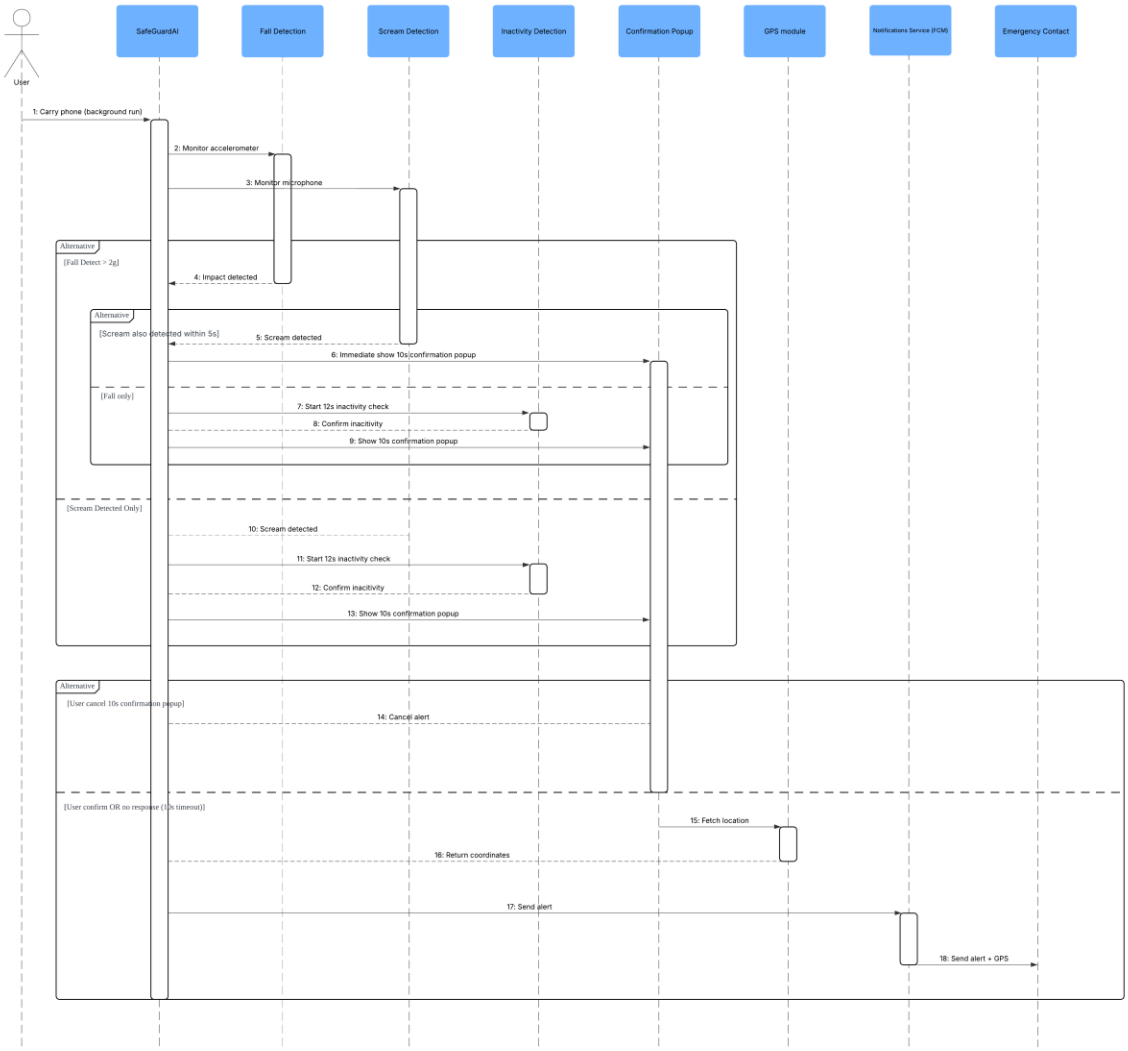


Figure 3.3.8: Automatic Detection Module Sequence Diagram

In automatic detection module, the process begins with continuous monitoring of the accelerometer and microphone. Then, when a fall is detected (impact >2g), the system will check for a scream followed by 5 seconds. If both detections occur, the app immediately shows a 10 second confirmation popup. However, if only a fall or scream is detected, then the system will start 12 inactivity checks to check immobility before showing confirmation popup. At this stage, the user can cancel the alert to avoid false alarm or confirm it/ let the 10s timeout to confirm the emergency. Furthermore, the SafeGuardAI will retrieve GPS location and send the alert to the predefined emergency contacts.

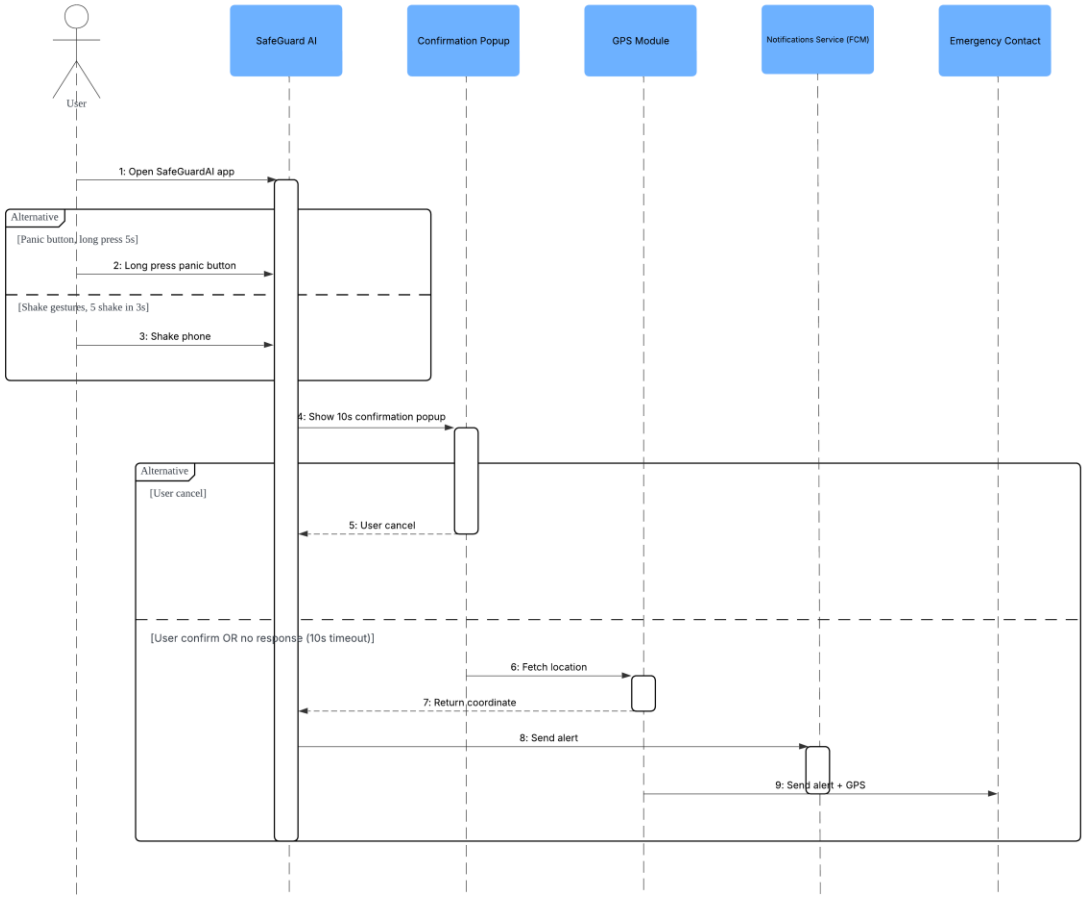


Figure 3.3.9: Manual Trigger Module Sequence Diagram

In manual trigger module, users can either press 5 seconds or shake it (5 shakes in 3s) to trigger emergency alerts. At these points, the SafeGuardAI will also show a 10s confirmation popup. If the user cancels it, no further action is taken. On the contrary, the app will send the alert and GPS to the predefined contact.

3.4 Functional and Non-Functional Requirements

This section specifies the functional and non-functional requirements for SafeGuardAI. The functional requirements are split into user requirements (what end users must be able to do) and system requirements (what the application must perform automatically). These requirements define the intended behavior of core features automatic multi-modal monitoring, manual SOS triggers, confirmation and notification flows, user consent, and event logging. Furthermore, the non-functional requirements establish quality attributes the system must meet, covering availability, usability, security, and cross-platform compatibility, with measurable targets (for example, backend availability ≥ 95% and clear privacy constraints such as no raw audio upload by default). Together these requirements provide a fundamental basis for implementation, validation, and later evaluation.

3.4.1 Functional Requirements

User Requirements

1. User should be able to enable or disable Auto-Safety mode from the settings screen.
2. User should be able to add, edit, and remove up to 2 emergency contacts.
3. User should be able to perform a manual SOS using a panic button (long-press activation).
4. User should be able to perform a manual SOS using a shake gesture (configured threshold/pattern).
5. User should be able to view a 10-second confirmation popup when an automatic or manual alert is triggered and cancel the alert within that time.
6. User should be able to see an alert history listing past alerts and alert type.
7. User should be able to configure basic settings including confirmation popup duration and panic-button hold time (within allowed ranges).
8. User should be able to receive notifications (push messages) when an alert is dispatched to their emergency contacts (confirmation of delivery status).
9. User should be able to grant or deny permission for on-device audio processing (user consent).

System Requirements

1. System should allow user to manage settings such as manage emergency contacts and toggle the Auto-Safety mode.
2. System should run background monitoring (accelerometer & audio) when the Auto-Safety mode is on.
3. System should detect a fall event when an impact spike (SMV threshold) is followed by post-impact stillness.
4. System should detect a scream event by classifying short audio snippets using on-devices features (MFCC) and produce a confidence score.
5. System should immediately show the confirmation popup when both fall and scream (defaults 10s).
6. System should start inactivity timer when only a single event is detected (default 12s), then show the confirmation popup when the user stays stillness and motionless for the setting timer.
7. System should allow user to cancel confirmation popup (default 10s) to minimize false alarm.
8. System should notify up to 2 emergency contacts after confirmation popup timeouts (user does not cancel it) and include alert type and live GPS coordinates.
9. System should obtain user permission to get their GPS location and audio before starting on-device processing and sharing locations.

3.4.2 Non-Functional Requirements

Availability

The system should be available whenever the user enables Auto-Safety. The app must maintain background monitoring and the ability to dispatch alerts during typical user activity, the backend services (notification routing & logging) should be available ≥ 95% of the time. This ensures that users can rely on the app during last-mile commutes and that alerts are deliverable when needed. (AltexSoft Editorial Team, 2023)

Usability

The system should provide a simple, friendly interface so users can trigger manual alerts quickly and understand confirmation popups. Furthermore, core UI elements (panic button, confirmation popup) should be reachable in one thumb motion and clearly labelled. Usability testing with target users should show that ≥ 90% of participants can trigger a manual SOS within 5 seconds in a test scenario.

Security

The system should prioritize user privacy: raw audio must not be uploaded to external servers and protect user data. Where any sensitive data is stored or transmitted (GPS coordinates in alerts), it must be minimized and encrypted in transit.

Compatibility & Portability

The system should run on both Android and iOS devices, providing consistent behavior while acknowledging platform differences (background execution constraints). The app should degrade gracefully where a platform restricts background sensor access and informs users if a function is limited on their device.

3.5 Chapter Summary and Evaluation

This chapter describes the methodology and requirement assessment for creating SafeGuardAI. The Incremental Model was selected as the project approach since it allows for the phased development and testing of the system (detecting falls, recognizing screams, monitoring inactivity, and notification features). This approach ensures that every component can be verified before integration, reducing risk and improving quality.

Requirements were gathered through 2methods: questionnaire and unstructured interviews. The questionnaire provided insights into user preferences, such as preferred confirmation frequencies, comfort with background audio management, and common travel habits. Then, the unstructured interviews improved this by collecting more detailed user insights on real safety needs. Together, these approaches helped verify that the system requirements align with the actual needs of users.

The requirement assessment was then conducted, and the results were categorized into functional and non-functional requirements. Functional requirements were divided into user requirements (manual triggers, alert history) and system requirements (fall detection logic, scream detection utilizing MFCC+SVM, inactivity timers, GPS-enabled alerts). Non-functional requirements centered around availability, usability, security, and compatibility, ensuring that the application is not just operational but also effective, safe, and reliable.

In the end, sequence diagrams were used to depict the real-time interactions between different elements of SafeGuardAI. These charts provided clarity on the order of occurrences for both automated detection and manual activation, enhancing the logical connections among modules and demonstrating how notifications are relayed to emergency contacts.

In summary, Chapter 3 established a solid foundation for system design and development by detailing the methodology, gathering and analyzing user requirements, and demonstrating the interaction of key features. This ensures that the next design phase (Chapter 4) can proceed with a clear and validated framework.

Chapter 4

System Design

4 System Design

This System Design introduces the overall design approach for SafeGuardAI. It provides high-level system design and how the system will be structured. This chapter covers topics such as use case diagram, UI design, data design (class diagrams), process design (activity diagrams), software architecture design, and security design. Each design shows how the system interacts with each other, and also ensuring the SafeGuardAI meets both functional and non-functional requirements.

4.1 Use Case Design

Figure 4.1.1: Use Case Diagram for SafeGuardAI

Use Case 1: Enable / Disable Auto-Safety

Use Case Name	Enable / Disable Auto-Safety
Objectives	To allow the user to manually activate or deactivate the automatic monitoring (fall detection, scream detection, inactivity detection) depending on their needs, ensuring flexibility and saving battery when not required.
Actor / Activator	User
Pre-Conditions	1. The app is installed and running on the user's smartphone.
Post-Conditions	Auto-Safety mode is turned ON / OFF, and the system updates background monitoring accordingly.

Basic Flows	
Actor	**System**
1. User navigates to the settings screen.	-
2. User taps the toggle button for Auto-Safety.	3. System updates the Auto-Safety mode (ON or OFF).
-	4. System enables or disables background monitoring for accelerometer, microphone, and inactivity sensors.
-	5. Confirmation message ("Auto-Safety Enabled" / "Auto-Safety Disabled") is shown to the user.

Alternate Flow
A1: If Auto-Safety mode, turn off
A1.1: The system stops all background monitoring.
A2: If device permissions (microphone, location) were not previously granted
A2.1: System prompts the user to grant permissions before enabling Auto-Safety.
A2.2: If user denies permissions, Auto-Safety cannot be activated, and an error message is displayed.

Use Case 2: View Alert History

Use Case Name	View Alert History
Objectives	To allow the user to review past alerts (automatic or manual) including type and location, so they can monitor their safety history and validate app behavior.
Actor / Activator	User
Pre-Conditions	1. The app is installed and running on the user's smartphone. 2. At least one alert has been recorded in the Firestore.
Post-Conditions	The user can see a list of past alerts with relevant details (alert type, date, time, GPS) from Firestore.

Basic Flows	
Actor	**System**
1. User navigates to the "Alert History" section.	-
2. User requests to view stored alerts.	3. System retrieves alert history data from the Firestore.
-	4. Firestore returns stored alerts logs.
-	5. System displays a list of alerts with type and GPS location.
6. User scrolls or taps an alert to view more details.	-

Alternate Flow
A1: If there is no alert history available
A1.1: Firestore return no records.
A1.2: The system displays a message: "No alerts have been recorded yet."
A2: Network error
A2.1: If the system cannot connect to Firestore, an error message is displayed: "Unable to retrieve alert history. Please check your connection."

Use Case 3: Grant Permissions (Mic, GPS)

Use Case Name	Grant Permissions (Mic, GPS)
Objectives	To request and obtain the necessary permissions (microphone for scream detection, GPS for alert location) from the user to enable SafeGuardAI safety monitoring and alert features.
Actor / Activator	User
Pre-Conditions	1. The app is installed and running on the user's smartphone. 2. Permissions for microphone and GPS have not yet been granted.
Post-Conditions	The system either gains the required permissions and enables the respective features or displays an error/warning message if permissions are denied.

Basic Flows	
Actor	**System**
1. User enables Auto-Safety mode or accesses a feature requiring permissions.	2. System checks current permission status (microphone, GPS).
2. User requests to view stored alerts.	3. If permission is missing, the system displays a prompt to request permission (OS dialog).
4. User reviews the permission prompt and selects "Allow."	5. System records granted permissions and enables relevant features (scream detection, GPS sharing).

Alternate Flow
A1: User Denies Permission
A1.1: System displays error message: "Permission required for SafeGuardAI to function."
A1.2: Auto-Safety or feature remains disabled until permission is granted.
A2: User Grants Partial Permissions (GPS but not Mic)
A2.1: A2.1: System enables only available features.
A2.2: Displays warning message: "Some features are disabled due to missing permissions."

Use Case 4: Fall Detection System (Impact + Stillness)

Use Case Name	Fall Detection System (Impact + Stillness)
Objectives	To automatically detect when the user experiences a potential fall by identifying a sudden impact followed by a period of stillness and then trigger a confirmation popup to verify if an emergency alert should be sent.
Actor / Activator	User (passive, monitored by system sensors)
Pre-Conditions	1. Auto-Safety mode is enabled. 2. Device sensors (accelerometer, gyroscope) are active and running in the background. 3. User has granted necessary permissions.
Post-Conditions	10-second confirmation popup is shown to the user if a fall event is detected. If the user does not cancel, the system proceeds with sending an alert.

Basic Flows	
Actor	**System**
-	1. System continuously monitors accelerometer data in the background.
-	2. System detects sudden impact exceeding the set threshold (impact >2g).
-	3. System verifies stillness for 12 seconds post-impact.
-	4. If criteria match fall event, system includes Show 10s Confirmation Popup use case.
5. User views the confirmation popup.	-
6. User either cancels or does not respond.	-
-	7. If no cancellation within 10s, system triggers Confirm / Timeout will Send Alert.

Alternate Flow
A1: False Fall Detected (User quickly moves again)
A1.1: System cancels detection as stillness criteria not met.
A1.2: No popup or alert is shown.
A2: User Cancels Alert within 10s Popup
A2.1: System executes Cancel Alert (extend) use case.
A2.2: No alert is sent, and the event may be logged for reference.

Use Case 5: Scream Detection (MFCC + SVM)

Use Case Name	Scream Detection (MFCC + SVM)
Objectives	To automatically detect distress screams or unusual loud sounds using microphone input and on-device AI, then trigger a confirmation popup if a scream is identified.
Actor / Activator	User (passive, monitored by system microphone)
Pre-Conditions	1. Auto-Safety mode is enabled. 2. Microphone permission is granted. 3. On-device ML model (MFCC + SVM) is available for classification.
Post-Conditions	A 10-second confirmation popup is shown to the user if a scream is detected. If the user does not cancel, the system proceeds with sending an alert.

Basic Flows	
Actor	**System**
-	1. System continuously records short audio snippets (1s) from the microphone.
-	2. System preprocesses audio (framing, filtering, normalization).
-	3. System extracts features using MFCC and classifies sound with SVM.
-	4. If scream is detected with confidence ≥ threshold, system includes Show 10s Confirmation Popup use case.
5. User views the confirmation popup.	-
6. User either cancels or does not respond.	-
-	7. If no cancellation within 10s, system triggers Confirm / Timeout will Send Alert.

Alternate Flow
A1: Non-Scream Detected (background noise, normal voice)
A1.1: System ignores event, no popup shown.
A2: User Cancels Alert within 10s Popup
A2.1: System executes Cancel Alert (extend) use case.
A2.2: No alert is sent, but the event may be logged for reference.

Use Case 6: Inactivity Monitoring (Stillness Timer)

Use Case Name	Inactivity Monitoring (Stillness Timer)

Objectives	To detect if the user remains motionless for a defined period (12 seconds) after a suspicious event, such as a fall or scream, and confirm whether assistance is needed.
Actor / Activator	User (passive, monitored by system accelerometer)
Pre-Conditions	1. Auto-Safety mode is enabled. 2. Microphone permission is granted. 3. Either a fall or scream event has been detected as a prerequisite.
Post-Conditions	If inactivity persists beyond the set timer, the system shows a 10-second confirmation popup. If no cancellation occurs, the system proceeds to send an alert.

Basic Flows	
Actor	**System**
-	1. System detects a prior trigger (fall or scream).
-	2. System activates stillness monitoring using the accelerometer.
-	3. System checks if user remains motionless (≤ 0.1 m/s²) for the timer duration (default 12s).
-	4. If stillness confirmed, system includes Show 10s Confirmation Popup use case.
5. User views the confirmation popup.	-
6. User either cancels or does not respond.	-
-	7. If no cancellation within 10s, system triggers Confirm / Timeout will Send Alert.

Alternate Flow
A1: User Resumes Movement Before Timeout
A1.1: Inactivity timer is canceled.
A1.2: No popup is shown, and monitoring returns to normal.
A2: User Cancels Alert within 10s Popup
A2.1: System executes Cancel Alert (extend) use case.
A2.2: No alert is sent, but the event may be logged.

Use Case 7: Manual SOS: Panic Button (5s long-press)

Use Case Name	Manual SOS: Panic Button (5s long-press)
Objectives	To allow the user to immediately request help by long pressing the panic button on the app's main screen for 5 seconds. This feature provides a direct emergency trigger if the user is conscious and able to act.

Actor / Activator	User
Pre-Conditions	1. The app is installed and running on the user's smartphone. 2. At least one emergency contact is predefined (up to 2 total).
Post-Conditions	An emergency alert is sent to the predefined contacts with the user's live GPS location if the user does not cancel the confirmation popup.

Basic Flows	
Actor	**System**
1. User navigates to the main screen.	-
2. User long-presses the panic button for 5 seconds.	-
-	3. System detects long-press input and activates Show 10s Confirmation Popup (include).
4. User views the popup and can cancel or ignore.	-
-	5. If no cancellation within 10s, system executes Confirm / Timeout will Send Alert.
-	6. System retrieves GPS coordinates.
-	7. System sends notification via FCM to up to 2 predefined contacts including alert type and live GPS.

Alternate Flow
A1: Short Press (<5s)
A1.1: No action is triggered, panic button requires a sustained press.
A2: User Cancels Alert within 10s Popup
A2.1: System executes Cancel Alert (extend) use case.
A2.2: No notification is sent.

Use Case 8: Manual SOS: Shake Gesture (5 shakes in 3s)

Use Case Name	Manual SOS: Shake Gesture (5 shakes in 3s)
Objectives	To allow the user to discreetly and quickly trigger an SOS alert by shaking the device five times within three seconds. This provides a hands-free and subtle activation method in emergencies where pressing the panic button is not possible.
Actor / Activator	User
Pre-Conditions	1. The app is installed and running on the user's smartphone. 2. At least one emergency contact is predefined (up to 2 total). 3. Motion sensors (accelerometer) are active.
Post-Conditions	If the shake gesture is correctly recognized and not cancelled, an emergency alert is sent to the predefined contacts with the user's live GPS location.

Basic Flows	
Actor	**System**
1. User navigates to the main screen.	1. System continuously monitors accelerometer input when Auto-Safety is enabled.
2. User long-presses the panic button for 5 seconds.	2. System detects a shake pattern (5 shakes in ≤ 3 seconds).
-	3. System activates Show 10s Confirmation Popup (include).
4. User views the popup and can cancel or ignore.	-
-	5. If no cancellation within 10s, system executes Confirm / Timeout will Send Alert.
-	6. System retrieves GPS coordinates.
-	7. System sends notification via FCM to up to 2 predefined contacts including alert type and live GPS.

Alternate Flow
A1: Incomplete Shake Gesture
A1.1: If fewer than 5 shakes are detected within 3 seconds, the system ignores the input.
A2: User Cancels Alert within 10s Popup
A2.1: System executes Cancel Alert (extend) use case.
A2.2: No notification is sent.

Use Case 9: Test Mode

Use Case Name	Test Mode
Objectives	To allow the user to practice the shake-gesture SOS trigger in a safe environment. This helps users understand how the manual shake detection works, while ensuring no alerts are sent to emergency contacts.
Actor / Activator	User
Pre-Conditions	1. The app is installed and running on the user's smartphone. 2. Test Mode is enabled in the settings.
Post-Conditions	The user successfully triggers a Test Shake Gesture Alert. The app confirms the action locally but does not notify any emergency contacts.

Basic Flows	
Actor	**System**
1. User enables Test Mode in settings.	-
-	2. System confirms Test Mode is active (visual indicator).
3. User performs the shake gesture (5 shakes within 3 seconds).	-
-	4. System detects the shake gesture.
-	5. System displays a confirmation popup (10s countdown) labelled as "Test Alert: No Notification Sent."
6. User views the popup.	-
-	7. System logs the test attempt into local alert history as "Test Mode: Shake Trigger."

Alternate Flow
A1: User Cancels the 10s Popup
A1.1: System cancels the test alert.
A1.1: Event logged as "Cancelled Test Shake Alert."
A2: User Forgets to Disable Test Mode
A2.1: System displays a reminder notification after 1 hour that Test Mode is still active.
A2.2: User must manually switch it OFF to resume real alert functionality.

4.2 UI Design

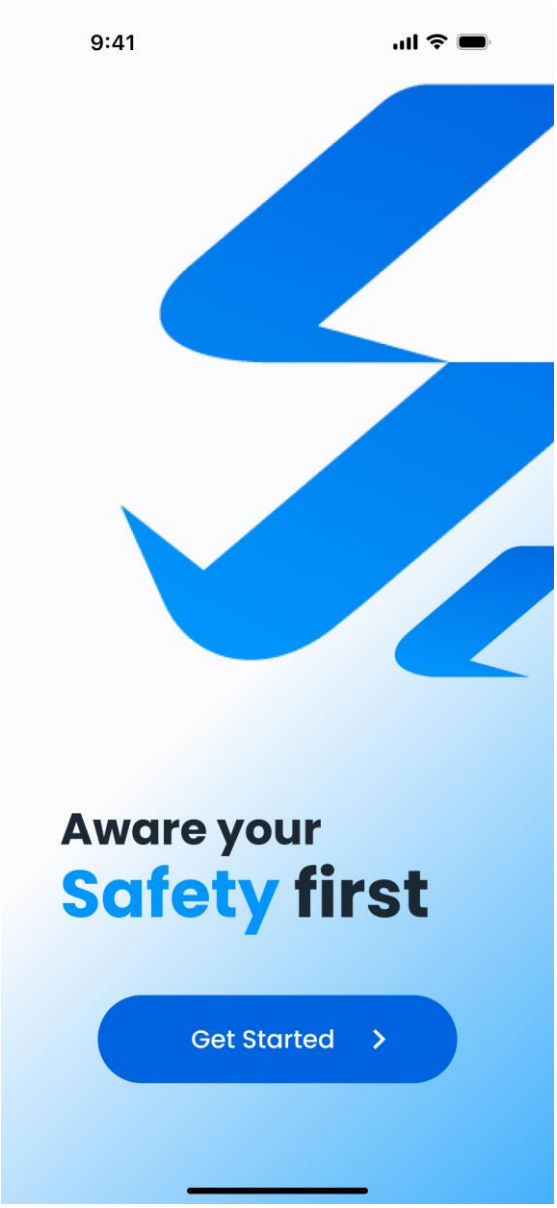


Figure 4.2.1: Start UI (Intro) Design

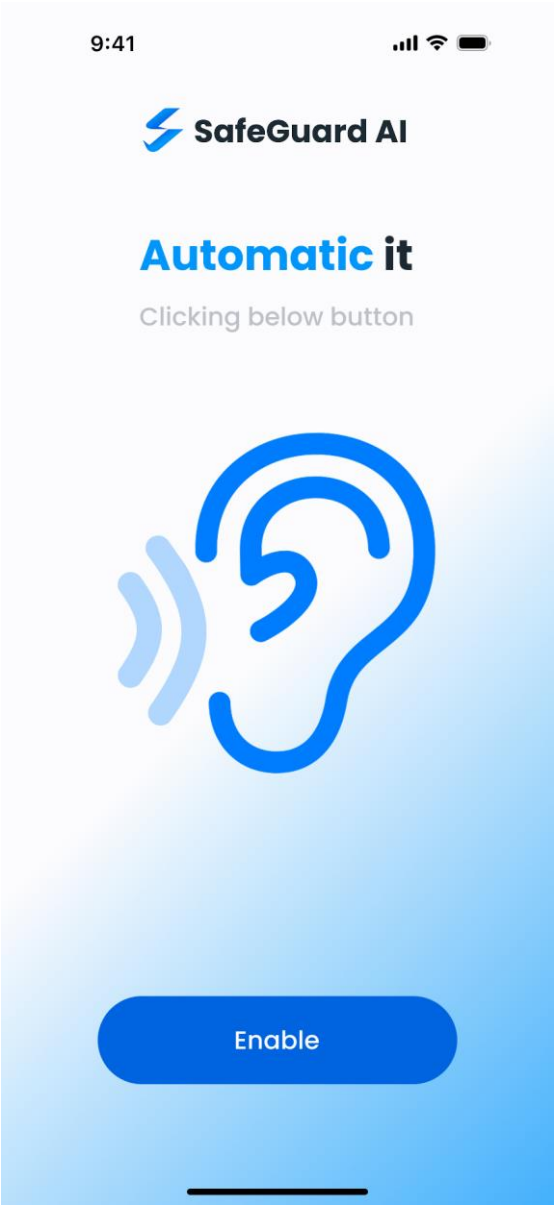


Figure 4.2.2: Enable Permission (Intro) UI Design

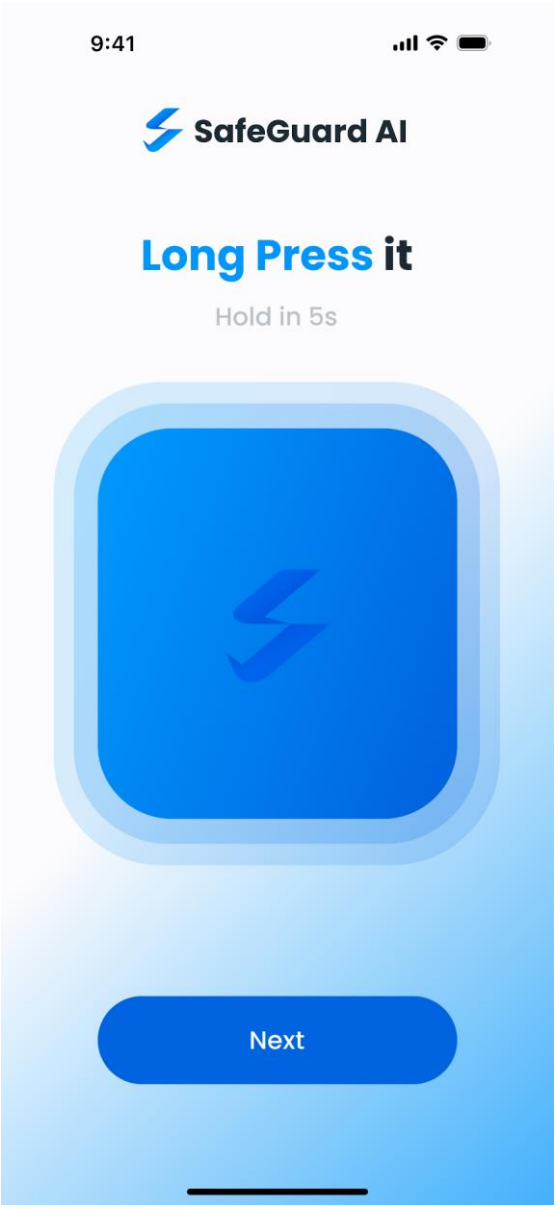


Figure 4.2.3: Long Press 5s (Intro) UI Design

Figure 4.2.4: Shaking 5x in 3s (Intro, Test Mode) UI Design

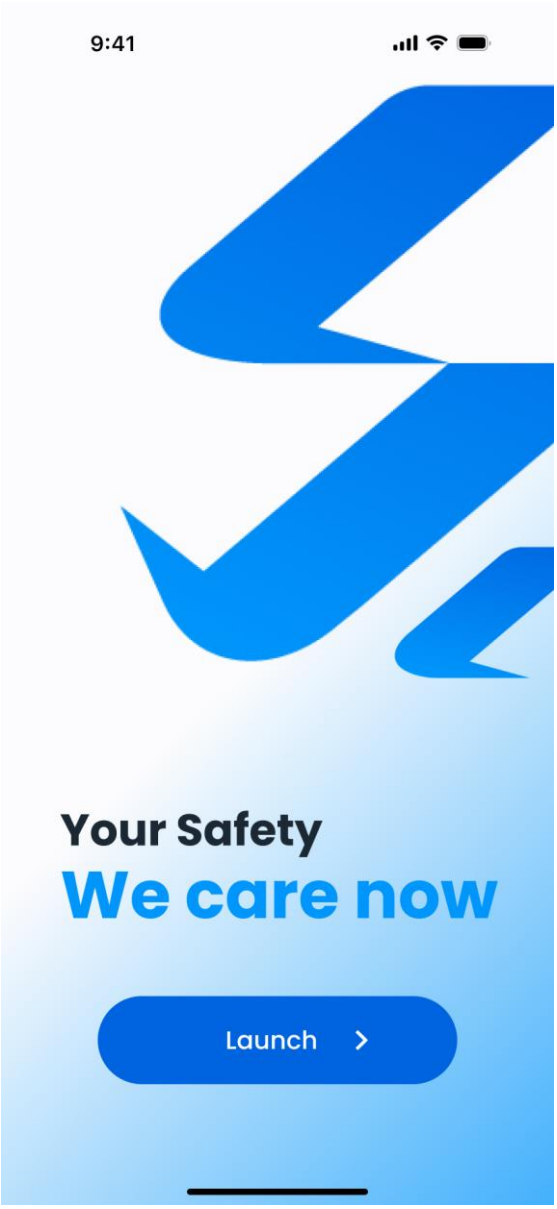


Figure 4.2.5: Done (Intro) UI Design

Figure 4.2.6: Home UI Design

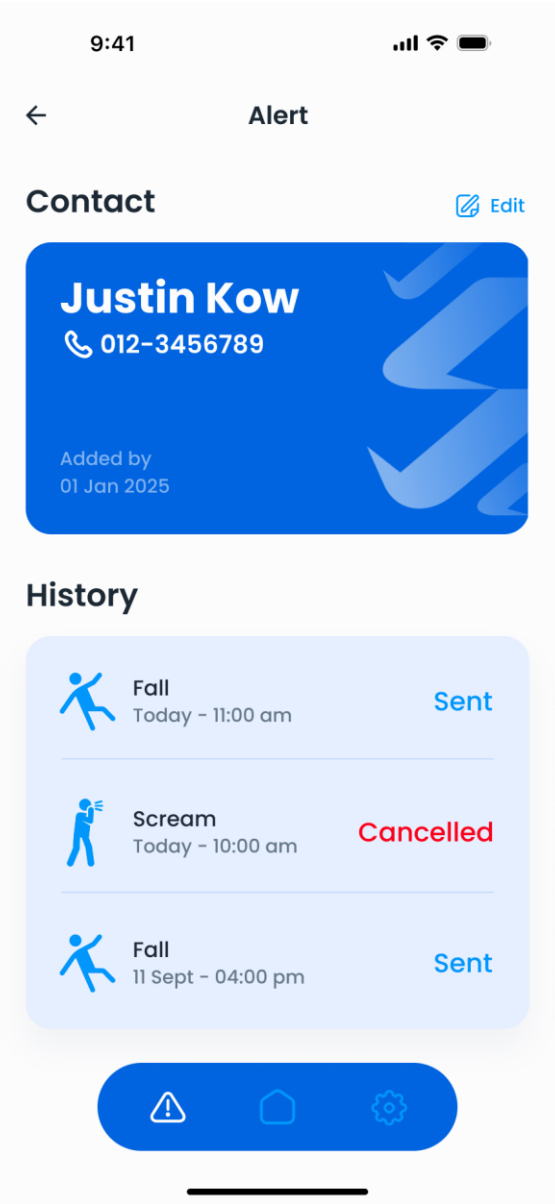


Figure 4.2.7: Alert & Emergency Contact UI Design

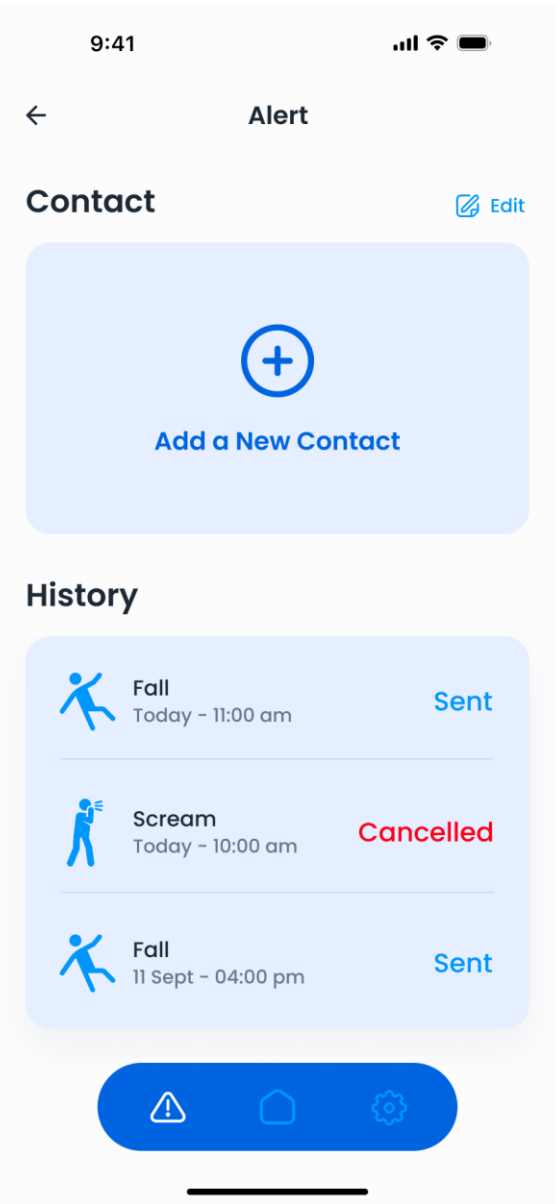


Figure 4.2.8: Alert & Emergency Contact (add) UI Design

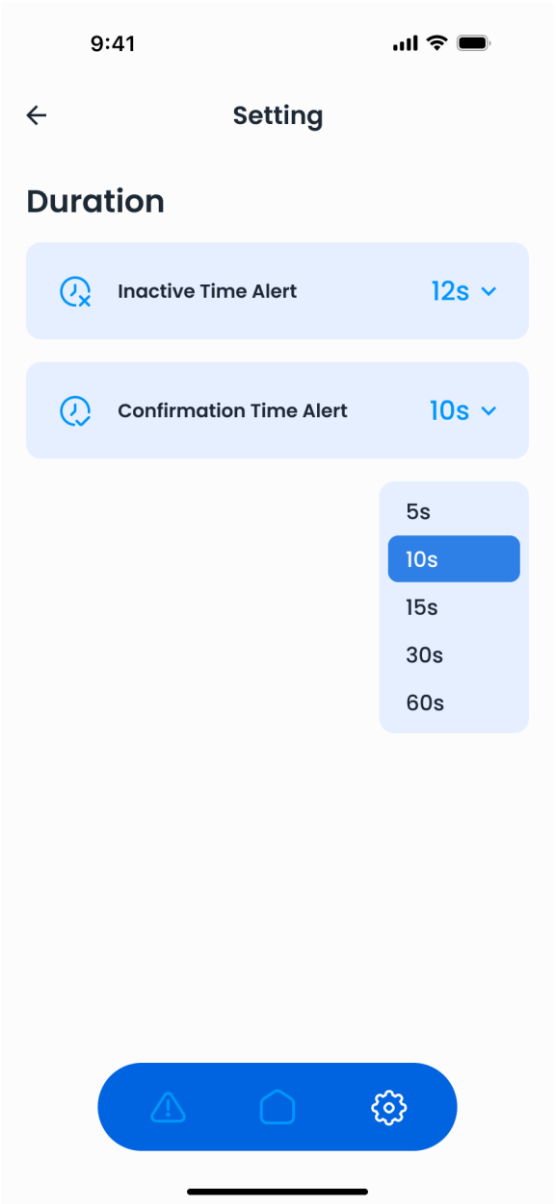


Figure 4.2.9: Setting UI Design

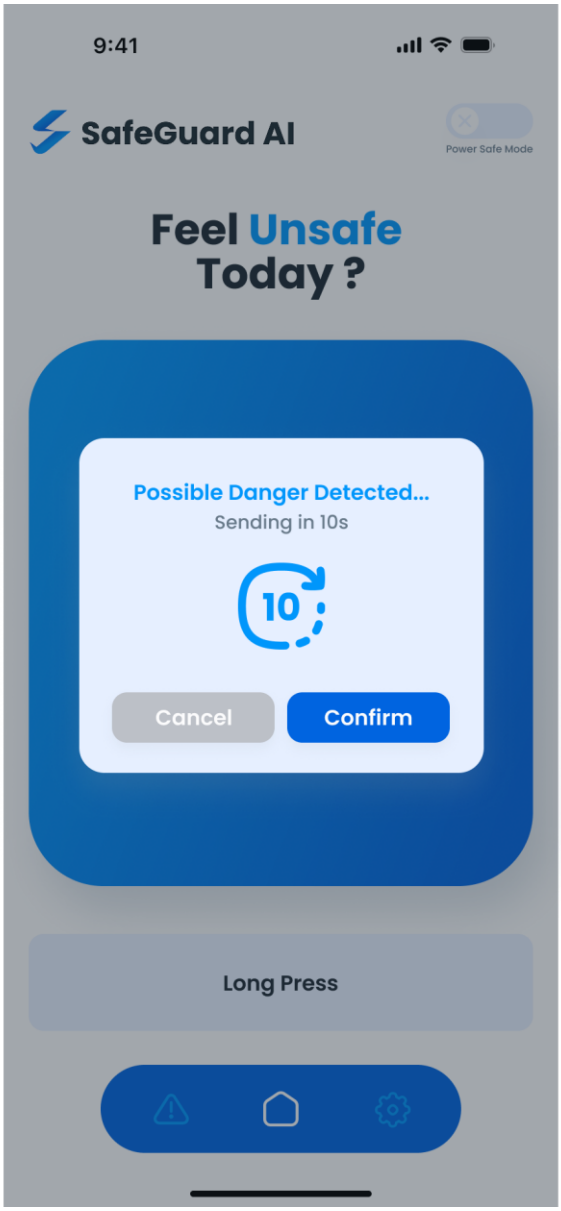


Figure 4.2.10: Confirmation Popup (10s) UI Design

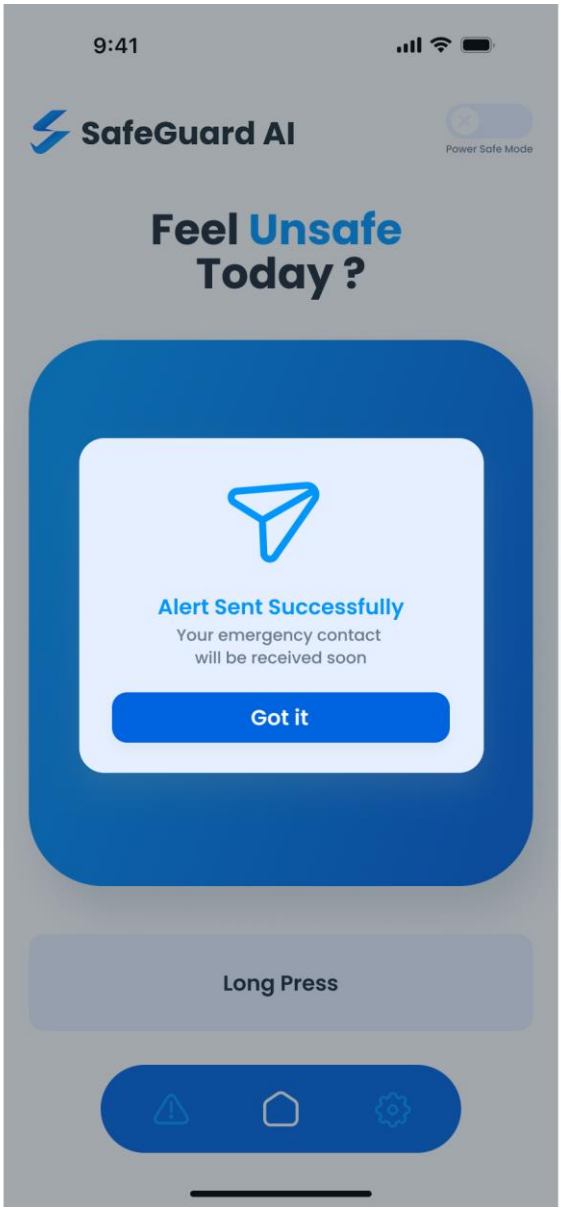


Figure 4.2.11: Successfully Sent Popup UI Design

4.3 Data Design

4.3.1 Class Diagram

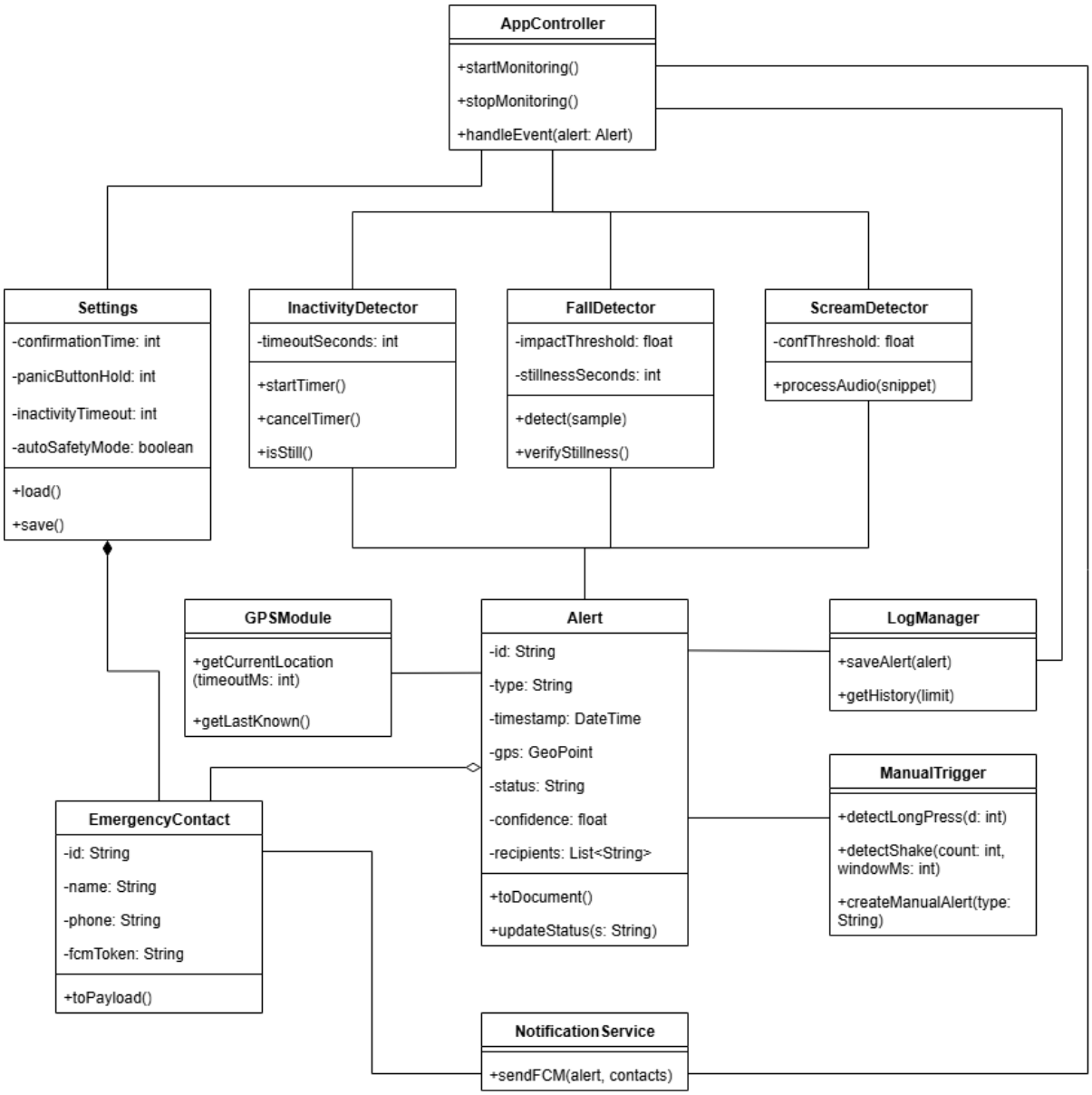


Figure 4.3.1: Class Diagram for SafeGuardAI

The class diagram for SafeGuardAI shows how the major components interact with each other. AppController serves as the main coordinator, responsible for starting and stopping the automatic monitoring and handling events that are generated by automatic detectors or manual triggers. Furthermore, the Setting class allows preference configuration. Each user can manage up to 2 emergency contacts only.

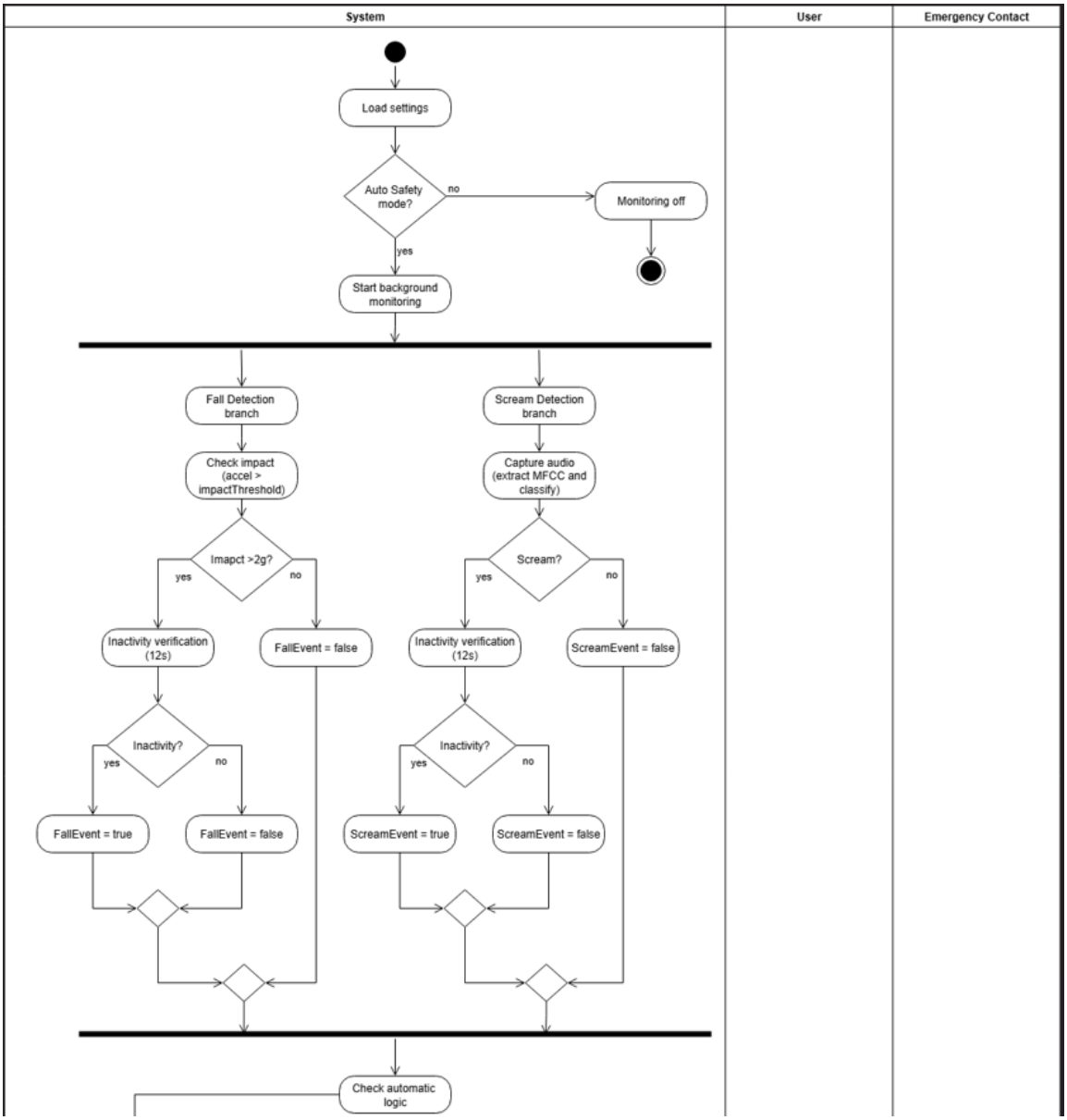
When an alert is detected, the Alert object is created. It records the important details such as alert type, timestamp, gps and so on. Alerts class will interact with GPSModule to retrieve

location data, with NotificationService to send messages through FCM and followed by logging history in LogManager.

Detection modules such as FallDetector, ScreamDetector and InactivityDetector using their own detection in accelerometer, audio and motion data. They will generate alert when the conditions are met. Moreover, the ManualTrigger class provides flexibility and alternative for user to raise alert through long-press or shake gestures.

4.4 Process Design

4.4.1 Automatic Detection Activity Diagram



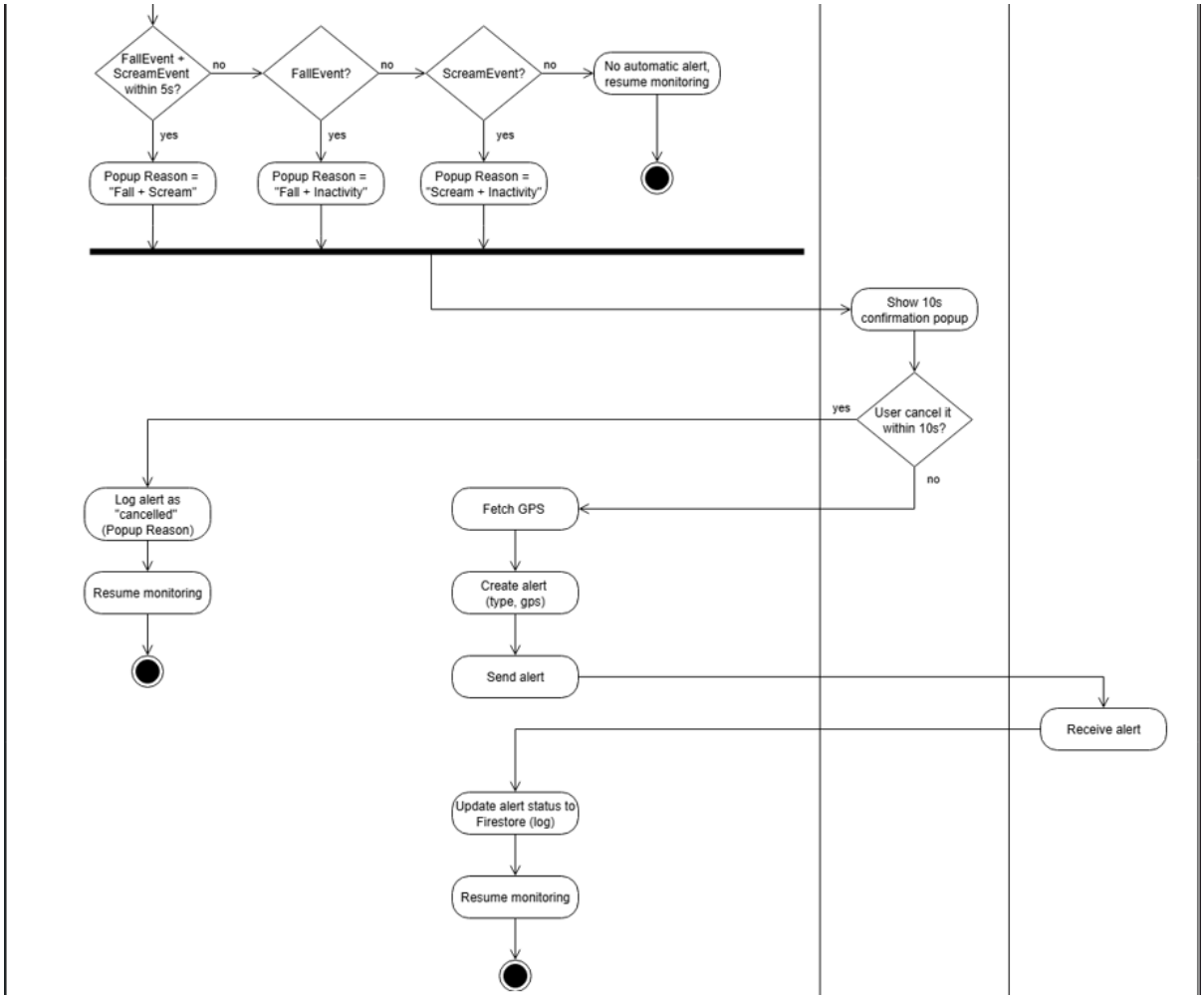


Figure 4.4.1: Automatic Detection Activity Diagram

The automatic detection activity diagram shows how SafeGuardAI continuously monitors the user's environment through the system. After Auto-Safety is activated, the system starts to monitor the accelerometer and microphone in the background. Simultaneously, the accelerometer detects a strong fall accompanied by a period of immobility, while the microphone analyzes brief audio segments utilizing MFCC to identify possible screams. Upon detecting a scream, the system initiates a stillness timer to verify lack of movement. The results are combined: when both a fall and a scream happen within 5 seconds, a confirmation popup appears immediately for 10 seconds to the user. Conversely, if solely a fall accompanied by inactivity or a scream with inactivity is identified, the identical popup is activated. At this stage, the user has ten seconds to cancel the alert; if they do, the event will be recorded as "cancelled" and the system will continue monitoring. If the user fails to respond, the system retrieves the GPS location, generates an alert record, stores it in Firestore, and alerts the emergency contacts through Firebase Cloud Messaging (FCM). This process maintains an equilibrium between responsiveness to genuine emergencies and the capacity to minimize false alerts.

4.4.2 Manual Trigger Activity Diagram

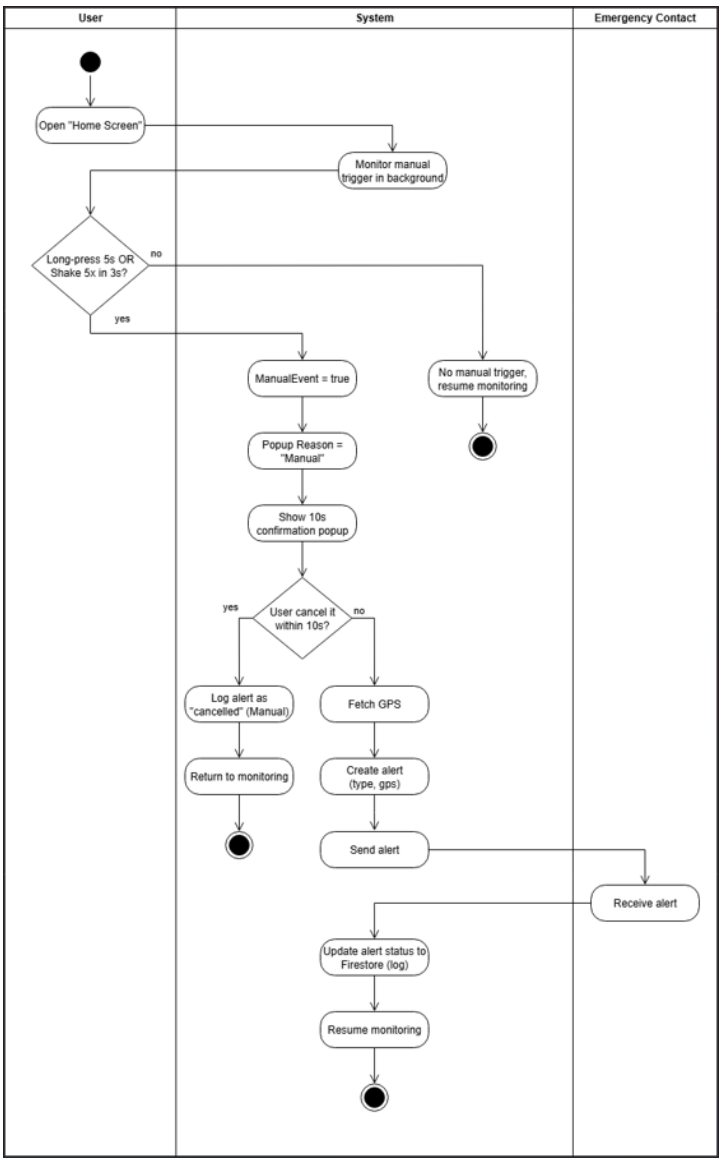


Figure 4.4.2: Manual Trigger Activity Diagram

The manual trigger activity diagram shows when users feel threatened, they can directly manually trigger it. There are 2 ways to activate it: by long pressing the panic button for 5 seconds, or by shaking the phone 5 times within 3 seconds. As soon as any of the actions is identified, the system shows a 10 second confirmation popup to the user. If the user decides to cancel, the alert is recorded as “cancelled” and monitoring persists. If the user affirms or there is no reply within the 10-second period, the system retrieves the GPS coordinates, generates an alert, records it, and sends notifications to the registered emergency contacts. This activity diagram highlights user agency by offering quick, dependable manual triggers that avoid the necessity for automatic detection, while still implementing the confirmation step to reduce unintentional activations.

4.5 Software Architecture Design

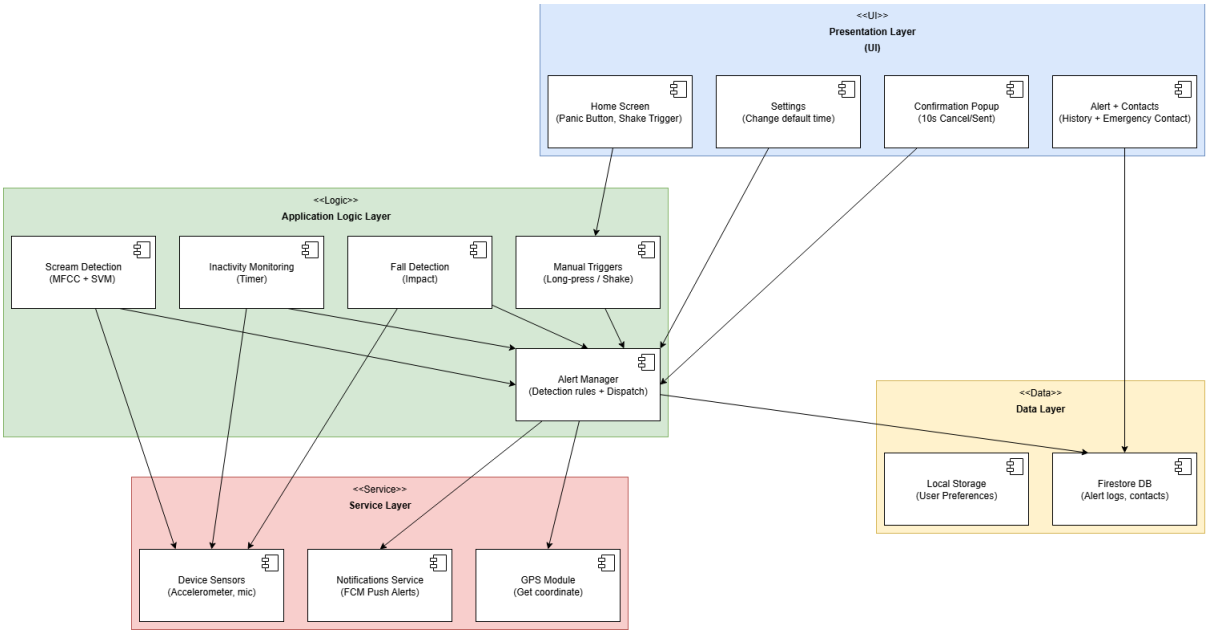


Figure 4.5.1: Software Architecture Design

SafeGuardAI software framework follows a layered design, splitting the system into four essential layers: the Presentation Layer, Application Logic Layer, Data Layer, and Service Layer. This framework fosters modularity, maintainability, and clear separation of concerns, improving the application's capacity to scale and adapt for future improvements.

The User Interface (UI) offers the interface that users interact with. It includes the Home screen (featuring a panic button, shake trigger, and Auto-Safety switch), the Alert & Contacts screen (for handling emergency contacts and checking alert history), the Settings screen (for setting inactivity timers and confirmation popup duration), and the 10-second confirmation popup (to either cancel or continue with alerts). These elements enable users to engage effortlessly with the system while maintaining a straightforward and user-friendly interface.

The Application Logic Layer contains the core modules dedicated to detection and alert supervision. This includes the Fall Detection element (impact), the Scream Detection element (employing MFCC + SVM classification), the Inactivity Monitoring element (timer-based stillness detection), and the Manual Triggers (long-press or shake). Every detection module links to the Alert Manager, which employs fusion rules (such as fall + scream, fall + inactivity) to determine whether to present the confirmation popup or dispatch alerts.

The Data Layer manages the oversight of information storage. It uses Firestore for recording alerts and keeping emergency contact details, while local storage serves for lightweight preferences such as user settings (confirmation period, Auto-Safety toggle, and inactivity timeout). This sector ensures that essential alert and contact information is protected, while user settings can be quickly retrieved offline.

This layered framework ensures that SafeGuardAI is robust, secure, and user-centered, with efficient communication between levels. By modularizing detection, storage, and services, the application minimizes complexity while maintaining flexibility for future improvements.

4.6 Security Design

The Security Design of SafeGuardAI prioritizes user privacy and the protection of sensitive data such as user GPS coordinates, audio snippets and emergency contact details. Therefore, SafeGuardAI uses privacy by design approach. All the sensitive processing such as Scream Detection (MFCC + SVM classification) is performed on device, ensuring the raw audio recordings are never uploaded to external servers.

Furthermore, all alerts and notifications sent to emergency contacts are being transmitted by Firebase Cloud Messaging (FCM) using TLS encryption to prevent data interception during transmission. Moreover, GPS coordinates are only fetched and attached to alerts when an emergency is confirmed, minimizing unnecessary exposure of location information. It also requests explicit user permissions for microphone and location access, and features such as Auto-Safety mode cannot be activated unless permissions are granted, reinforcing transparency and consent.

Data persistence is handled carefully (alert history and emergency contact details) are stored in Firestore, where access is controlled through Firebase Authentication and Firestore security rules. User preferences, such as inactivity timeout and confirmation popup duration, are stored securely in local storage, which avoids unnecessary backend traffic and reduces risks of data leaks.

In summary, the on-device AI, user consent enforcement and data persistence help SafeGuardAI ensures that user safety is protected without compromising privacy or exposing sensitive information to unnecessary risks.

4.7 Chapter Summary and Evaluation

In summary, this chapter shows the design and diagram in the SafeGuardAI. The use case diagram shows the core functionalities from the perspective of user. Furthermore, UI design provides a clear layout of how user interacts with the app. The activity diagrams detailed the logic behind both automatic and manual alert processes, ensuring no ambiguity in workflows. The layered software architecture design demonstrated how the system is structured into presentation, logic, data, and service layers, promoting maintainability and scalability. Finally, the security design highlighted essential measures to protect user data, including encryption, permissions, and secure communication channels.

Overall, the chapter ensures that the proposed design is consistent, feasible, and aligned with both user needs (from questionnaires and interviews) and technical requirements. This forms a strong foundation for implementation in later phases.

Chapter 5

Implementation and Testing

5 Implementation and Testing

A short introduction that describes what will be included in this chapter.

IMPORTANT NOTE TO STUDENTS: Include details about:

- **Implementation**

  A detailed description of how you actually carried out the implementation (e.g., coding, etc.) of your system/prototype.

  - Include code snippets and descriptions to show how the requirements of the application/prototype have been met –
    - For Smart Campus projects, code snippets of the MQTT protocol for both client side and server side has to be included with explanation.
  - Include descriptions of important settings - Setting for server, network protocol, IP, database, security
  - Note: this should **not** be a chronological account of the work you carried out.

- **Testing**

  All test cases that have been carried out must be provided - To tabulate the test cases in tables

Note: Students may also opt to split Implementation and Testing into 2 separate chapters.

5.1 Sub-section 1 Heading

Sub-sections should be used to divide the chapter into logical parts.

5.1.1 Sub-subsection Heading

Sub-section numbering should be limited to a maximum of 3 levels (e.g. 5.3.1) in order to avoid confusion.

5.2 Sub-section 2 Heading

5.3 Sub-section 1 Heading

Sub-sections should be used to divide the chapter into logical parts.

5.3.1 Sub-subsection Heading

Sub-section numbering should be limited to a maximum of 3 levels (e.g. 5.3.1) in order to avoid confusion.

5.4 Sub-section 1 Heading

Sub-sections should be used to divide the chapter into logical parts.

5.4.1 Sub-subsection Heading

Sub-section numbering should be limited to a maximum of 3 levels (e.g. 5.3.1) in order to avoid confusion.

5.5 Chapter Summary and Evaluation

At the end of each chapter, evaluate the contents stated or discussed in the relevant sub-sections.

Chapter 6 *(if applicable)*

System Deployment

6 System Deployment

This chapter would be applicable for students who have embarked on a real-life industrial project. Students in this case would need to describe how the deployment has been carried out. Some of implementation tasks which need to be described include: training, file conversion or creation, and changeovers.

System Backup and Risk Management

Describe the procedures to backup the existing system for changeover purpose. Discuss the potential risk(s) for the changeovers and the solution.

On-site Setup

Describe the preparations to be done prior to the setup of the new system on client's site. Discuss the procedures to setup the new system, schedule, etc.

Training Procedure

Describe the procedures on the training procedures, contents, schedule, etc.

Follow-up

Describe the plan or procedures to follow up with the client (company) to verify the system reliability.

Chapter Summary and Evaluation

At the end of each chapter, evaluate the contents stated or discussed in the relevant sub-sections. For example,

- Problems faced. Describe the various problems faced by students in the course of doing the project.
- Solutions. What have been done to solve the problems?
- What tools and techniques have been used and reasons for using them.

Chapter 7

Discussions and Conclusion

7 Discussions and Conclusion

Each student is required to make an *evaluation of the project* he/she has embarked on. The project evaluation may include the following sections.

IMPORTANT NOTE TO STUDENTS: In this chapter, for problems related to code, hardware, internet connection, etc (where applicable):

- List the technical problems faced and state how they were resolved
- List the unsolved technical problems for future enhancement
- List the achieved objectives/modules
- List the incomplete parts for future enhancement - this is regardless the parts listed in the pre-determined scope. Suggestions for future improvement

Summary

Summarize the project including the problem and proposed solutions, justification of the choice of tools, techniques and methodologies used in this project.

Achievements

Students are required to evaluate the project's achievement against project objectives, completion of the project, students' view of the strengths and weaknesses of the work done.

Contributions

Discuss the creativity, innovativeness, contribution of the proposed system. Explain why the proposed system is necessary. Describe the marketability of the system.

Limitations and Future Improvements

Identify the limitations of the research or project. Provide suggestions for improvement or further development of the system or research in the future.

Issues and Solutions

Students are required to describe the various problems faced by students during the project development and explain what has been done to solve the problems. The valuable experiences gained or lessons learnt through the project as a whole. The issues may include technical issues, project management issues, team dynamics problems, and other difficulties encountered and lessons learnt. How the issues are solved or can be solved to ensure the project can be completed on time or to be improved in the future.

References

1. *JABATAN PERANGKAAN MALAYSIA DEPARTMENT OF STATISTICS MALAYSIA*. (n.d.). https://storage.dosm.gov.my/crime/crime_2023.pdf
2. *The Ultimate Guide to Firebase Cloud Messaging (FCM) - Novu*. (2024, March 29). https://novu.co/blog/the-ultimate-guide-to-firebase-cloud-messaging-fcm/
3. Bicchielli, C. (2021). Multi-Sensor fall detection for smartphones. *Biomedical Journal of Scientific & Technical Research*, *34*(4). https://doi.org/10.26717/bjstr.2021.34.005588
4. IJRPR International Journal of Research Publication and Reviews. (n.d.). *International Journal of Research Publication and Reviews*. International Journal Publication Fees Under 500 Rs, Fast Publication, High Impact Factor Journal, Publication Charge 14 US Dollars. https://ijrpr.com/uploads/V6ISSUE5/IJRPR45730.pdf
5. Vreča, J., Pilipović, R., & Biasizzo, A. (2024). Hardware–Software Co-Design of an audio feature extraction pipeline for machine learning applications. *Electronics*, *13*(5), 875. https://doi.org/10.3390/electronics13050875
6. *What is the influence of C in SVMs with linear kernel?* (n.d.). Cross Validated. https://stats.stackexchange.com/q/159051
7. *Reddit - The heart of the internet*. (2021). Reddit.com. https://www.reddit.com/r/GalaxyWatch/comments/r4ix7c/testing_fall_detection_on_gw4/
8. *Touch & hold delay - Android Accessibility Help*. (n.d.). https://support.google.com/accessibility/android/answer/6006989?hl=en
9. *Try QA*. (n.d.). https://tryqa.com/what-is-incremental-model-advantages-disadvantages-and-when-to-use-it/
10. AltexSoft Editorial Team. (2023, December 30). Nonfunctional requirements in software engineering: Examples, types, best practices. *AltexSoft*. https://www.altexsoft.com/blog/non-functional-requirements/
11.

publication, etc.), should also be given. Web references can be listed separately (e.g., after the reference list) under a different heading if desired, or can be included in the reference list. The list of references should contain all the items that have been explicitly referred to in the report. Students may have

referred to some manuals or conducted research on certain knowledge areas, for example Oracle, RFID scanners etc.

If the students choose to write a bibliography, then they should list down all materials that are useful either directly or indirectly to the project concerned and which the students have read. An example of material useful could be an article or an Oracle user manual etc.

References should be cited in the main text using Author-Year system. The full references should be given as below (essentially APA Referencing format), in the alphabetical order in 10 pt. Times New Roman, with a 6pt spacing between each. References should be arranged first alphabetically and then further sorted chronologically if necessary. More than one reference from the same author(s) in the same year must be identified by the letters "a", "b", "c", etc., placed after the year of publication.

Please ensure that every reference cited in the text is also present in the reference list (and vice versa). Any references cited in the abstract must be given in full.

Web references used as a minimum, the full URL should be given and the date when the reference was last accessed. Any further information, if known (DOI, author names, dates, reference to a source

Appendices

In order to enhance better understanding of the project, students should as far as possible include all directly relevant materials, figures or diagrams in the **main body** rather than in the Appendix. The appendix is reserved only for items which may not directly be relevant or essential to enhance a reader's understanding of the project, or which may interrupt the smooth reading of the project document (for example being too voluminous).

Appendices should only include supportive materials **directly referred** to in the writing and should be kept to a **minimum**, e.g. selected pages of an annual report, not the entire document. Examples of items included in Appendices are:

- Company's report and documentation, such as sample invoice, purchase order form, etc.
- Project meeting documentation e.g. minutes of meetings, tracking documents, memos etc.
- Questionnaires and results, interview questions and results, pilot test and results, observation sheet and results, experiment test plan and results, etc.
- Analysis/design diagrams (only those not incorporated in the main body of the report).

If there is more than one appendix, they should be identified as A, B, etc (e.g. Appendix A). Formulae and equations in appendices should be given separate numbering: Eq. (A.1), Eq. (A.2), etc.; in a subsequent appendix, Eq. (B.1) and so on. Similarly for tables and figures: Table A.1; Fig. A.1, etc.

IMPORTANT NOTE TO STUDENTS

APPENDIX *n* User Guide

- List the username and password of multiple roles (if applicable)
- Provide clear screen shots of each page, explain the functions of each button

As a rough guide, the user guide should include the following sections:

System Document

In this section, students should provide the following pieces of information:

- **System (hardware and software requirements)**. Students should describe the minimum hardware and software requirements to install the software application which has been developed by the students, for example, DBMS, OS, program development tools etc.
- **Installation**. Under this section, students should create an 'installation' CD and provide a brief step-by-step guide on how a new user can install the software on a computer system. Students should indicate any special setup information, such as the specific location of placing the database files, the SQL statements to add tables, etc. Software source code should also be included in this CD. Students are not required to print out the software code.

Operation Document

Under this section, students are required to provide a brief step-by-step guide on how to use the installed software. The guides should teach the user how to run the software and use its major functions and features. For example, steps show guide the users on how to run the system, e.g. to use an executable file or to use the IDE. The login information such as username and password for each of the different users (or roles) must be provided. Some screen interfaces would be useful.

APPENDIX *n+1* Developer Guide*

*To be included for ***Smart Campus Projects*** and ***Real-Life Projects***. This section should include the following:

- List the necessary software, installer, API, library that must be installed
- List the authentication details, such as username and password for all security

Other Appendices:

- Supporting documents
- Non-disclosure agreement (if applicable)
- Other detailed documentation, such as important references, interview results, survey results, etc.

This page is intentionally left blank to indicate the back cover. Ensure that the back cover is black in color.